

UNIT-I

INTRODUCTION TO NEURAL NETWORKS

INTRODUCTION TO NEURAL NETWORKS: THE BIOLOGICAL NEURON, THE PERCEPTRON, MULTILAYER FEED-FORWARD NETWORKS, TRAINING NEURAL NETWORKS: BACKPROPAGATION AND FORWARD PROPAGATION ACTIVATION FUNCTIONS: LINEAR, SIGMOID, TANH, HARD TANH, SOFTMAX, RECTIFIED LINEAR, LOSS FUNCTIONS: LOSS FUNCTION NOTATION, LOSS FUNCTIONS FOR REGRESSION, LOSS FUNCTIONS FOR CLASSIFICATION. HYPERPARAMETERS: LEARNING RATE, REGULARIZATION, MOMENTUM, SPARSITY, GRADIENT DESCENT, VANISHING AND EXPLODING GRADIENT.

ARTIFICIAL NEURAL NETWORK

- ❑ An **Artificial Neural Network(ANN)** may be defined as an **information – processing model** that is inspired by the **Biological Nervous System** , such as **Brain**.
- ❑ The model tries to **replicate only** the most **basic functions of the Brain**.
- ❑ An **ANN** is composed of a **large no. of highly inter connected processing units (Neurons)** working to solve the **specific Problem**.
- ❑ Like **Human Brain**, ANN is also a **Learn by Example**.

- Just as we use our brains to identify patterns and classify different types of information
- By using the Neural Networks, it allowing computers to observe, learn, and react to complex situations faster than humans.
- An ANN is configured as different types of Applications, such as Face Recognition, Pattern Recognition etc. through a learning process.

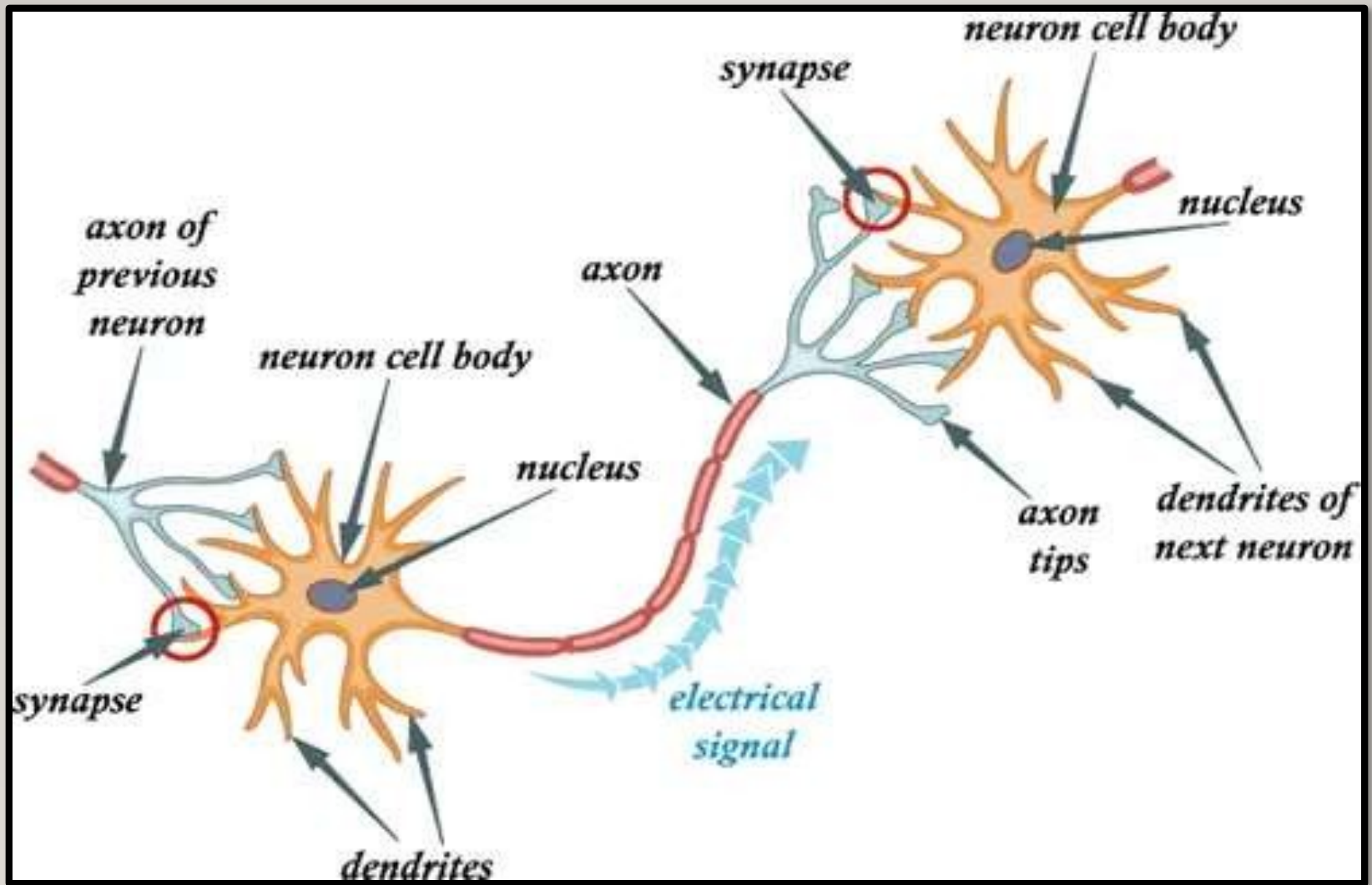
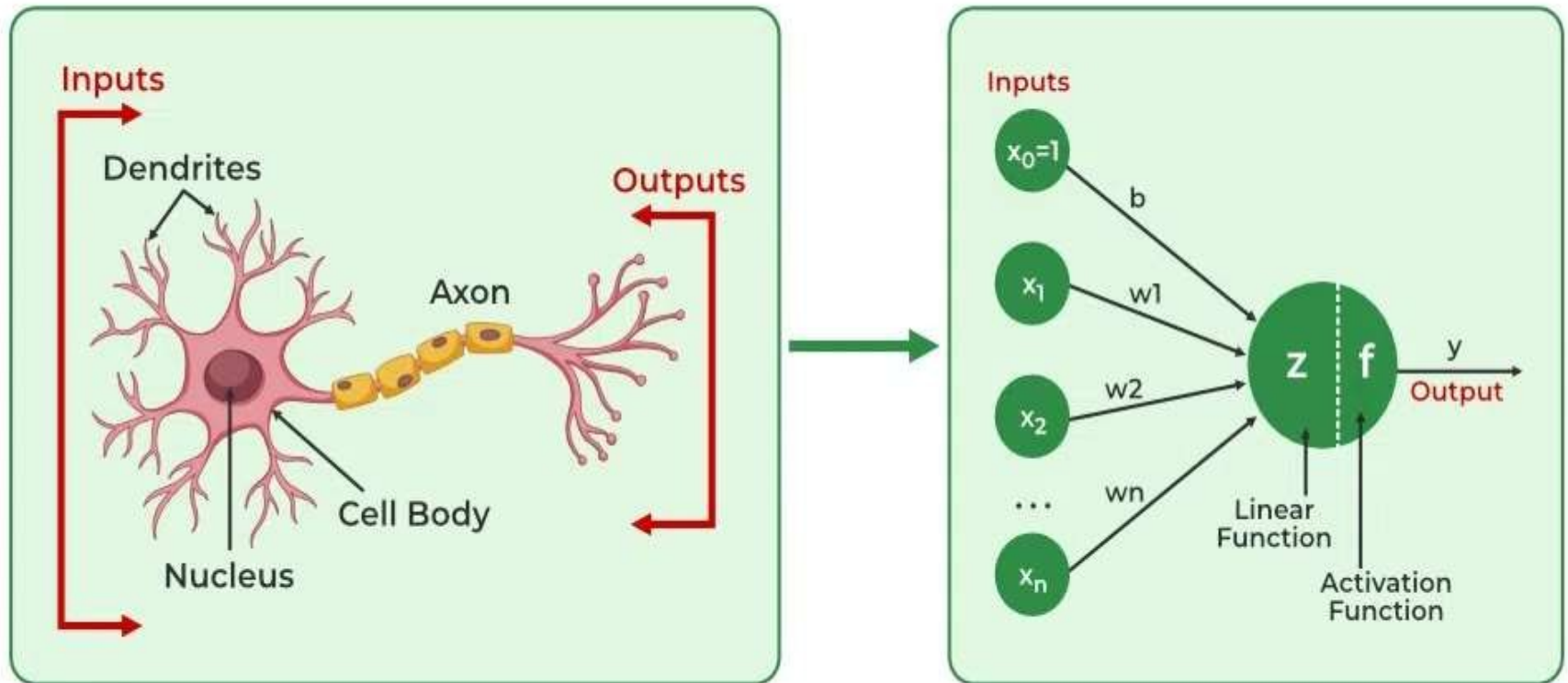


Figure: Biological Neural Network

- **Soma**: This is also called the **cell body**. It is where the **cell nucleus is located**.
- **Dendrites**: These are **tree-like networks** that are connected to the **cell body**. ie, it **receives the signal from other Neurons.(Input)**
- **Axon**: Axon carries the **signal from the cell body**. It splits into **strands (Chain or a single unit of a molecule such as DNA)** and each strand ends in a bulb-like structure called **synapse. (output)**
- **ie, a single axon** (the cell's output—carrying information away). **Synapse** is a junction between Two Neurons.



Biological Neural Network	Artificial Neural Network
Dendrites	Inputs
Cell nucleus	Nodes
Synapse	Weights
Axon	Output

Table: Terminology b/w Human Brain and ANN



ARCHITECTURE OF ANN

- ❑ Artificial Neural Networks are **processing elements** either in the form of **algorithms or hardware devices modeled**.
- ❑ Artificial Neural Network primarily consists of **Three Layers**:
 1. **Input Layer**
 2. **Hidden Layer**
 3. **Output Layer**
- ❑ **Each Layer** consists of **more Neurons**
- ❑ The **nodes of the Input Layer** can take **input data and perform operations on it and send the results** of these operations to **other neurons of the Hidden Layer**.
- ❑ The **hidden layer** sends **data to the Output Layer**.

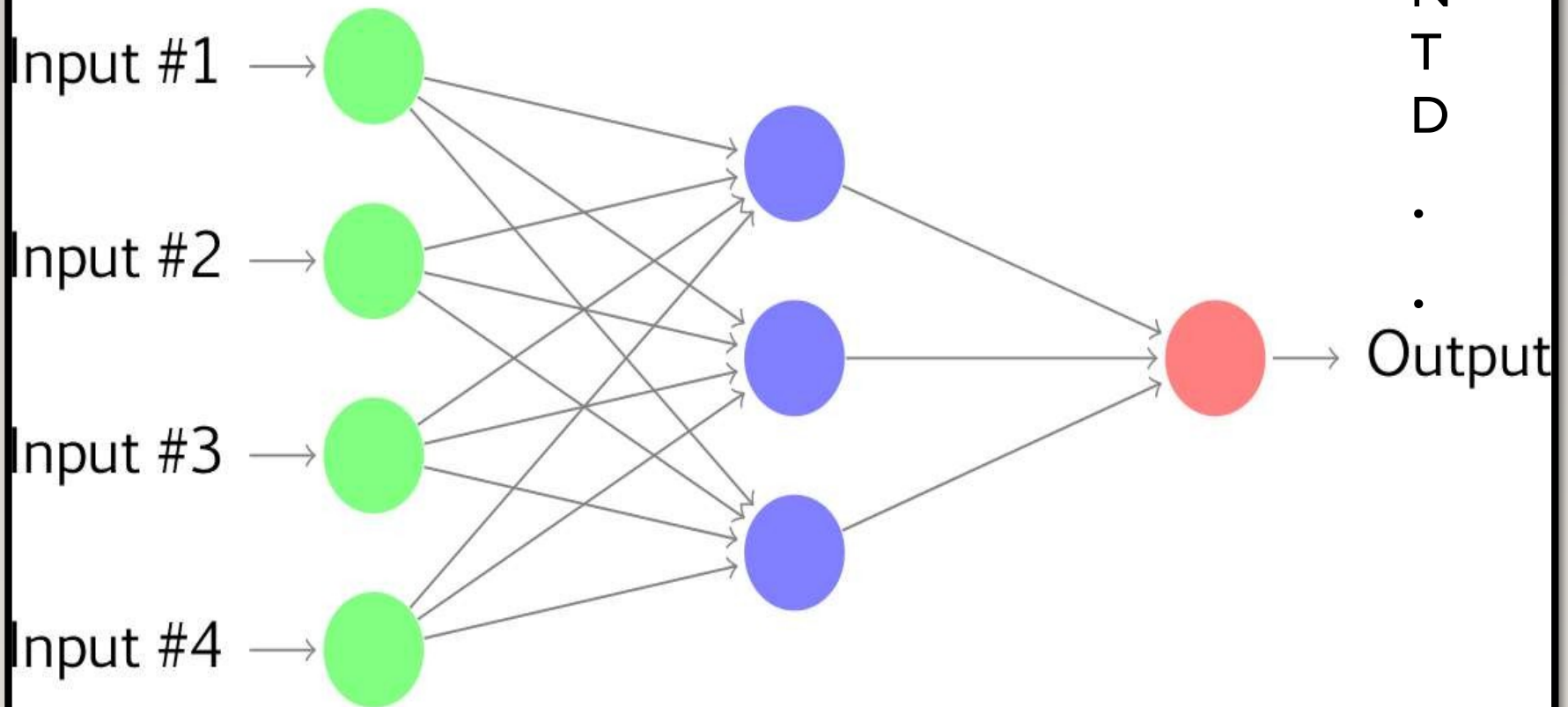
9

Input
layer

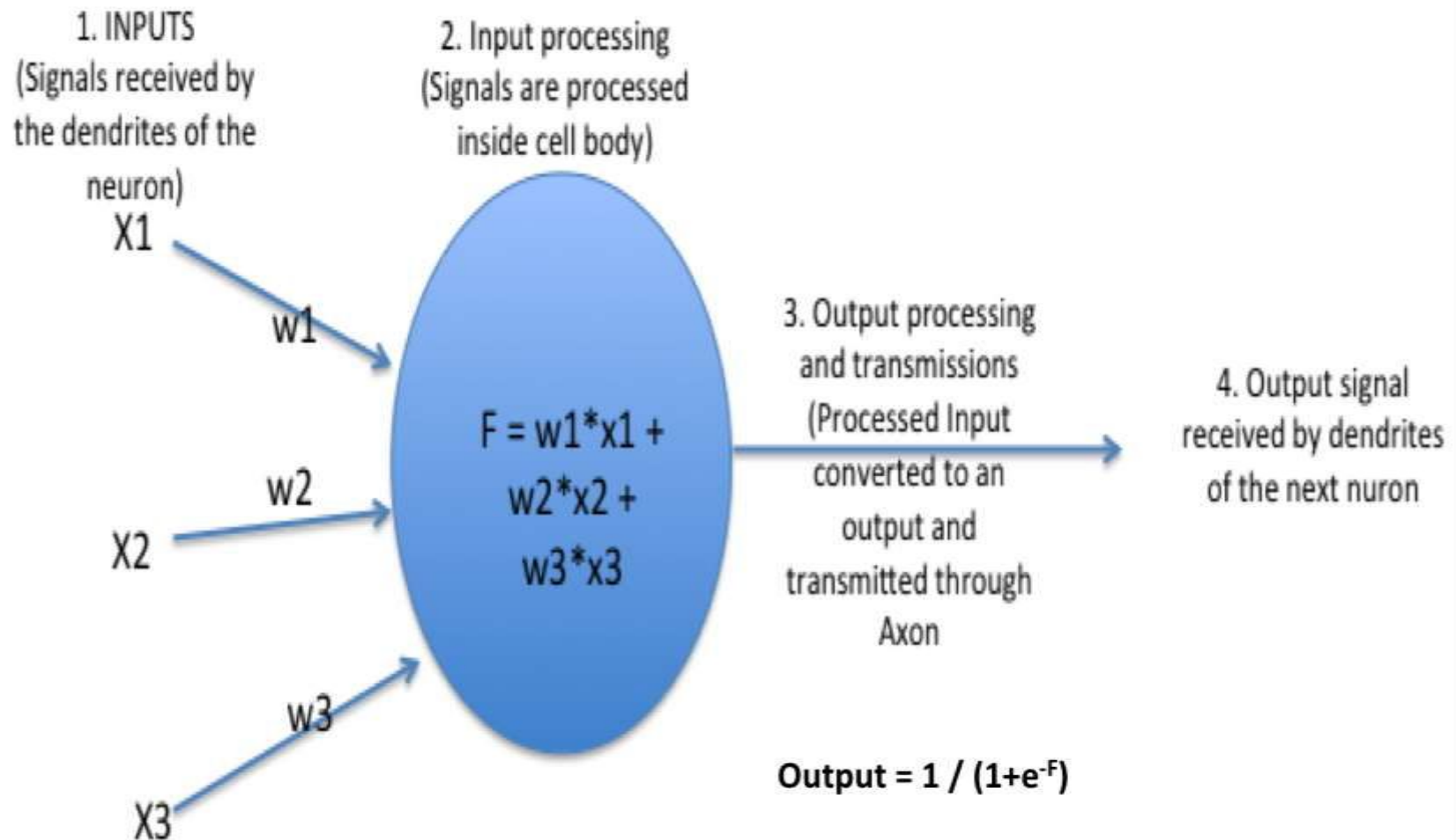
Hidden
layer

Output
layer

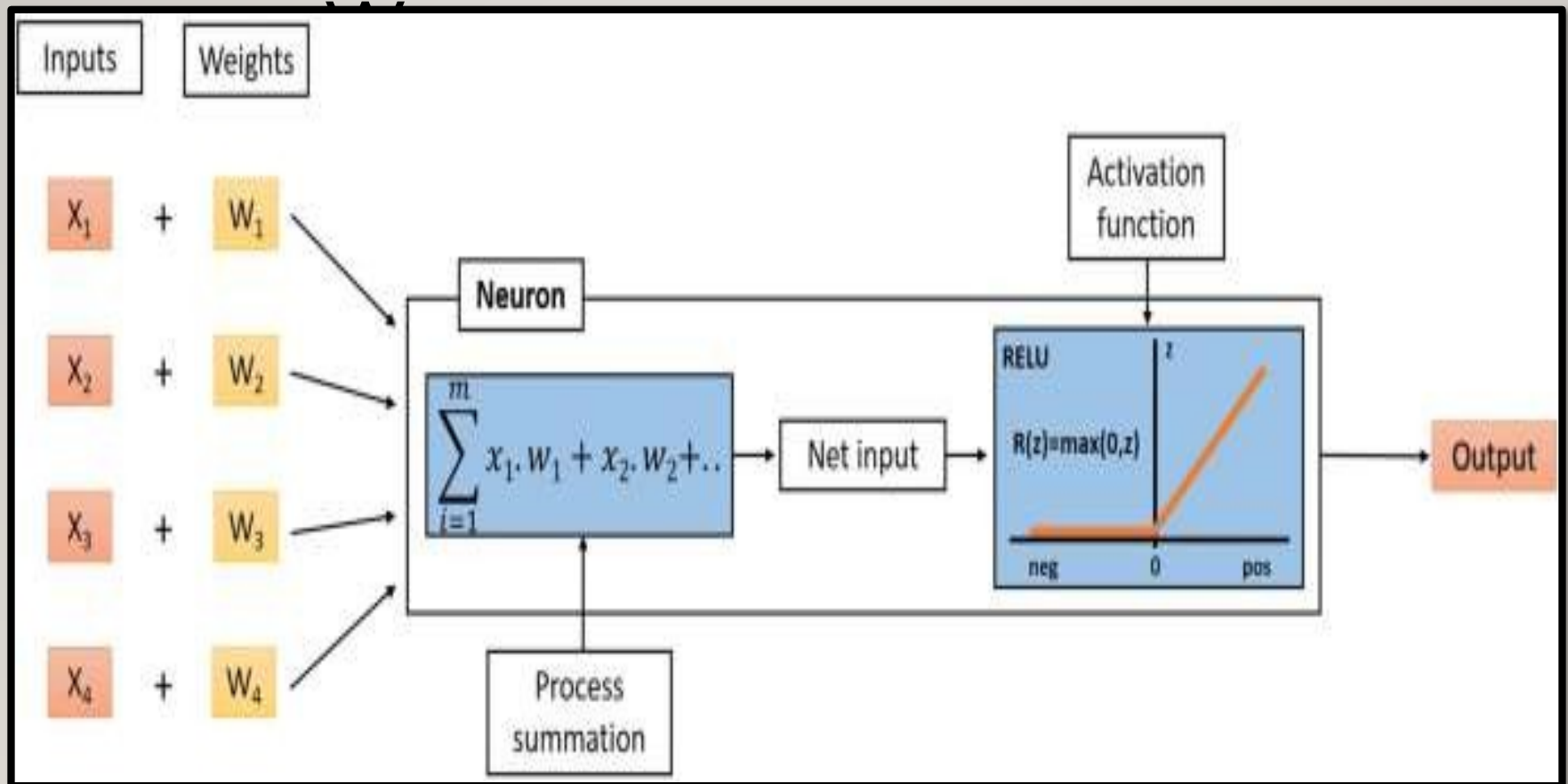
C
O
N
T
D



WHAT HAPPENS INSIDE THE NEURON



WHAT HAPPENS INSIDE THE NEURON

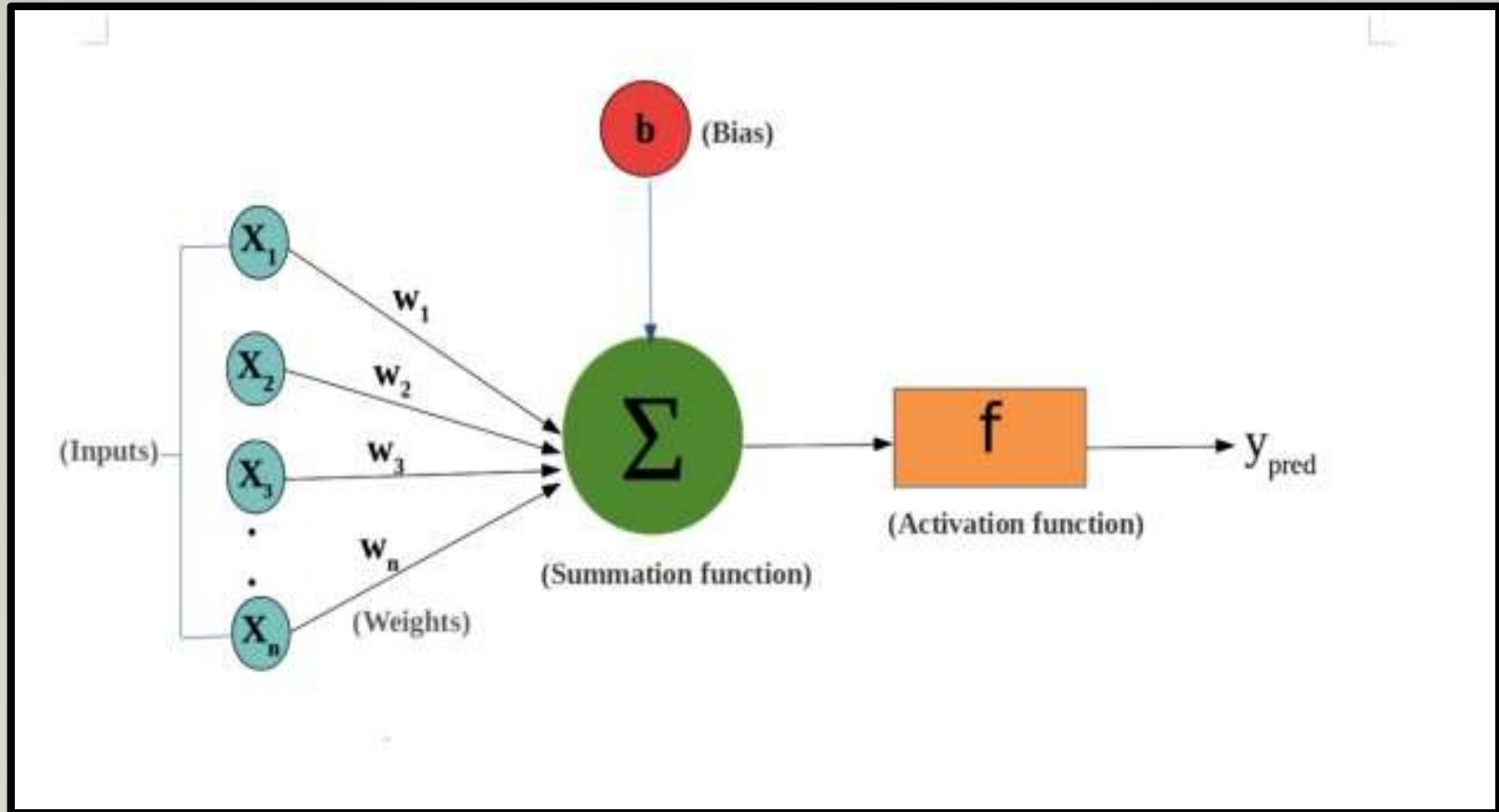


BIAS IN NEURAL NETWORK

- **Bias** is a **systematic error** that occurs due to **wrong assumptions** in the machine learning process.
- **Bias** is simply defined as the **inability of the model** because of that there is some **difference or error** occurring between the **model's predicted value** and the **actual value**.
- These differences between **actual or expected values** and the predicted values are known as **error or bias error or error due to bias**.
- **Biases** provide a **critical additional layer** of flexibility to **neural networks**. Biases are essentially constants associated with each neuron. Unlike weights, biases are not connected to

- **Biases** provide a **critical additional layer** of flexibility to **neural networks**.
- Biases are essentially **constants** associated with **each neuron**.
Unlike weights, **biases are not connected to specific inputs** but are **added to the neuron's output**.
- It is used to **adjust the output** along the **weighted sum of the inputs of the neuron**.
- This **Bias Value** allows you to **shift the Activation Function** to **either Left or Right**.

C



$$\text{output} = \text{sum} (\text{weights} * \text{inputs}) + \text{bias}$$

- we can represent the **sigmoid activation function** with the **mathematical expression** as

$$\text{sigmoid function} = \frac{1}{1 + e^{-x}}$$

replacing x with the equation of a line

$$\text{sigmoid function} = \frac{1}{1 + e^{-(w*x+b)}}$$

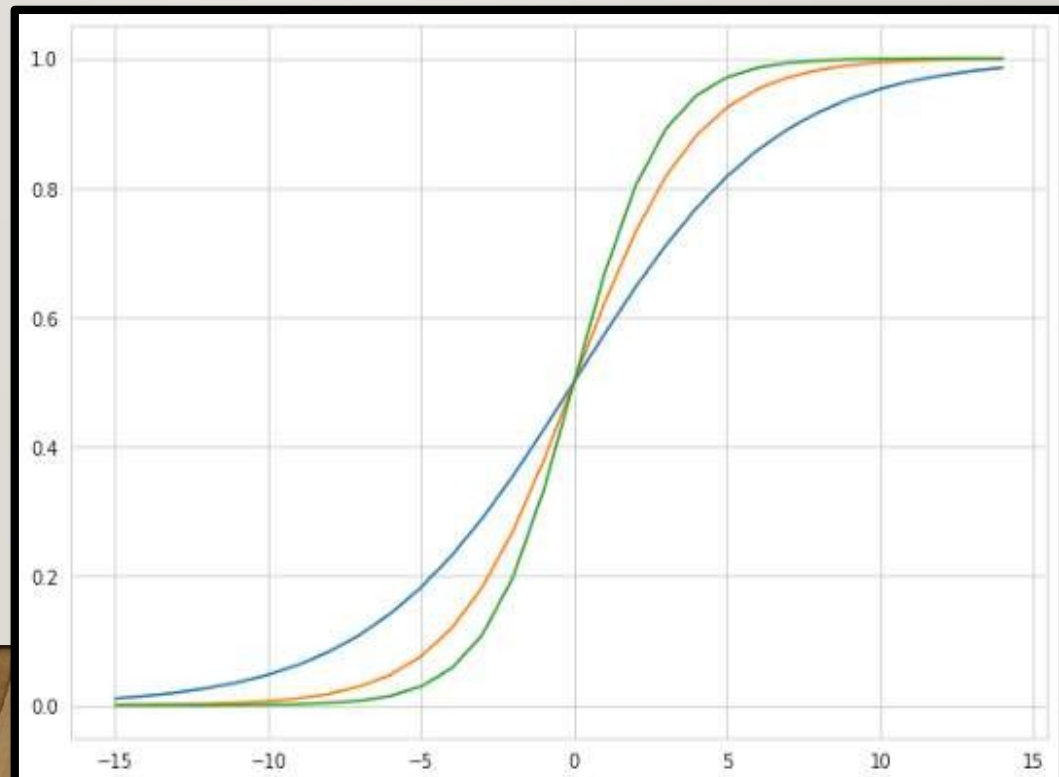


1. Let us vary different values of w and fix the b value to 0

when the **value of $b = 0$** and
 $w = 0.3$ — the blue line in the plot

$w = 0.5$ — the red line in the plot

$w = 0.7$ — the green line in the plot



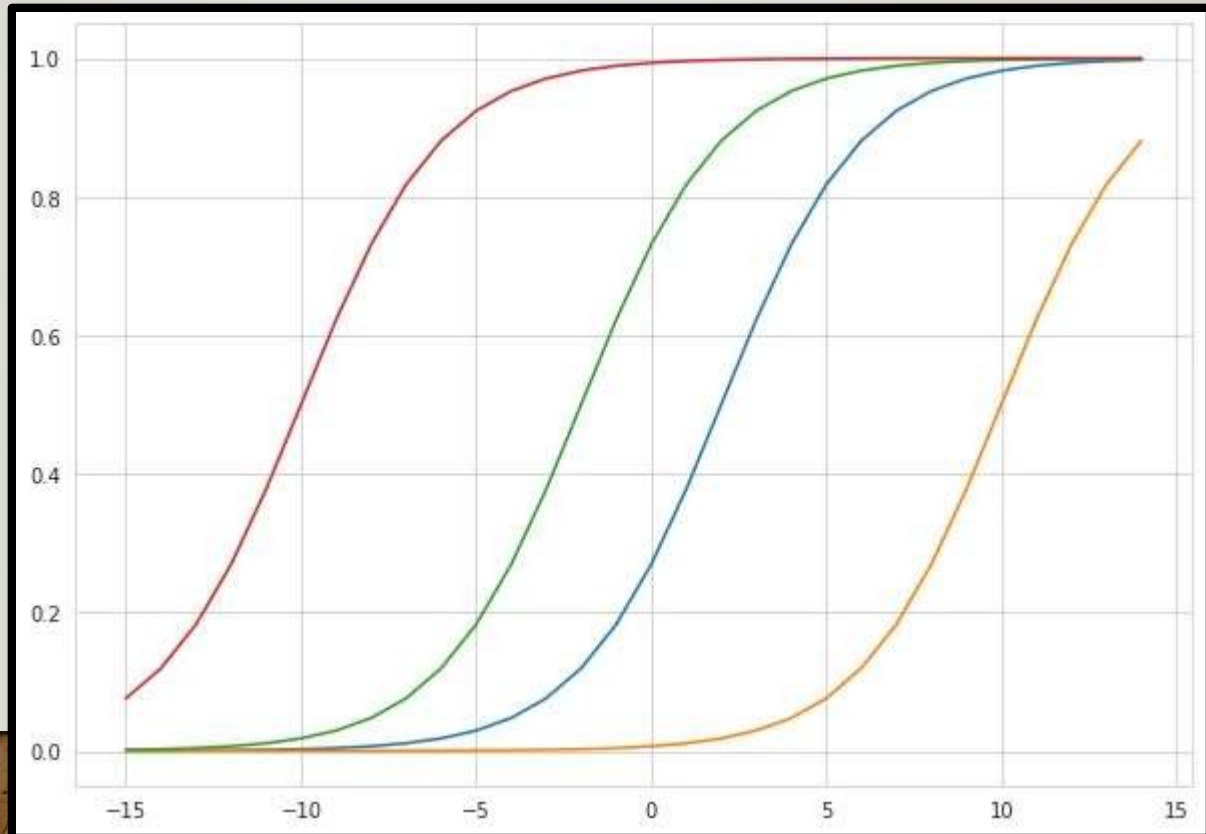
2. Let us vary different values of b and fix w value to 0.5

$b = -1$ — the red line in the plot

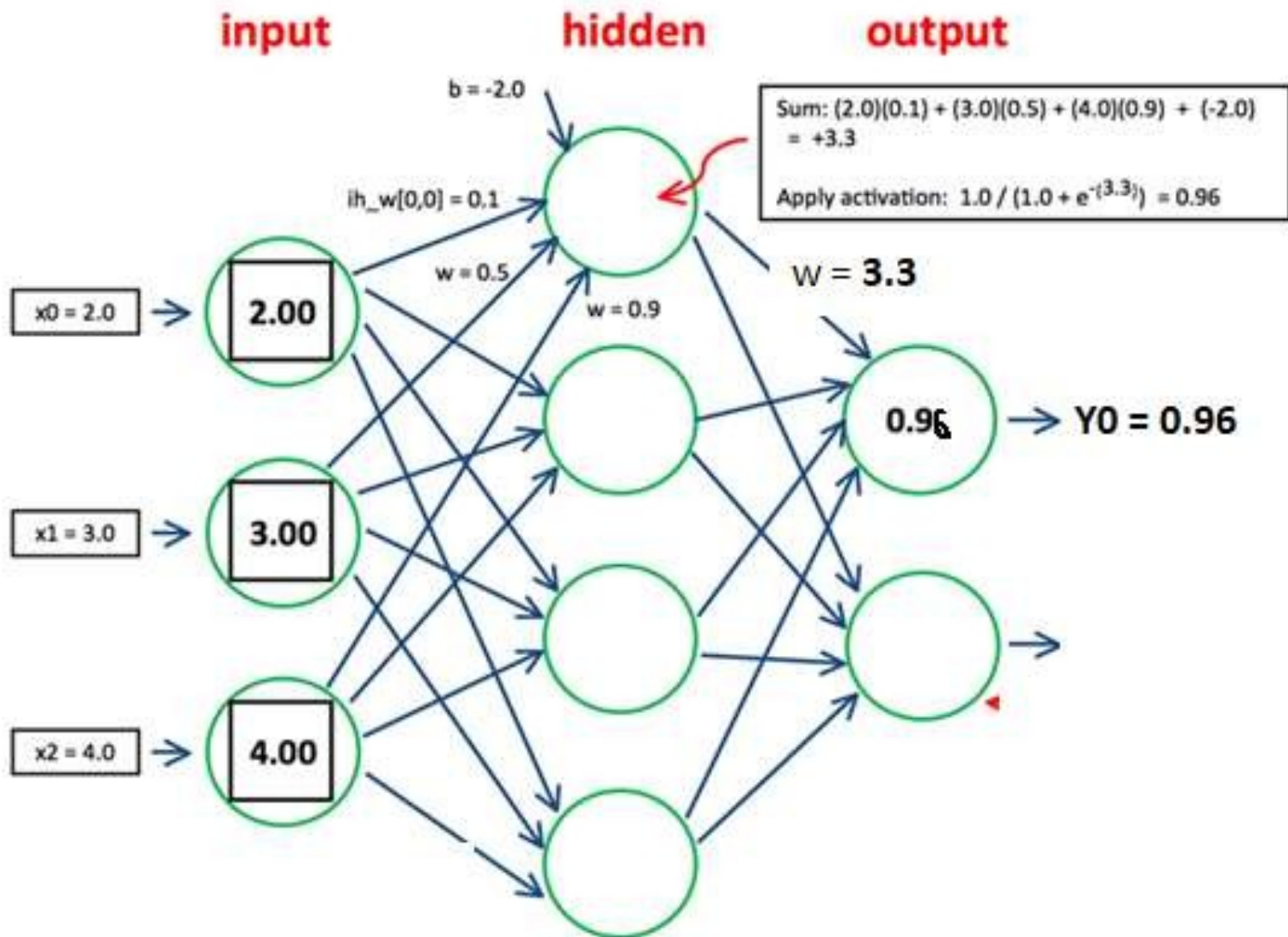
$b = -5$ — the green line in the plot

$b = 1$ — the blue line in the plot

$b = 5$ — the yellow line in the plot



EXAMPLE

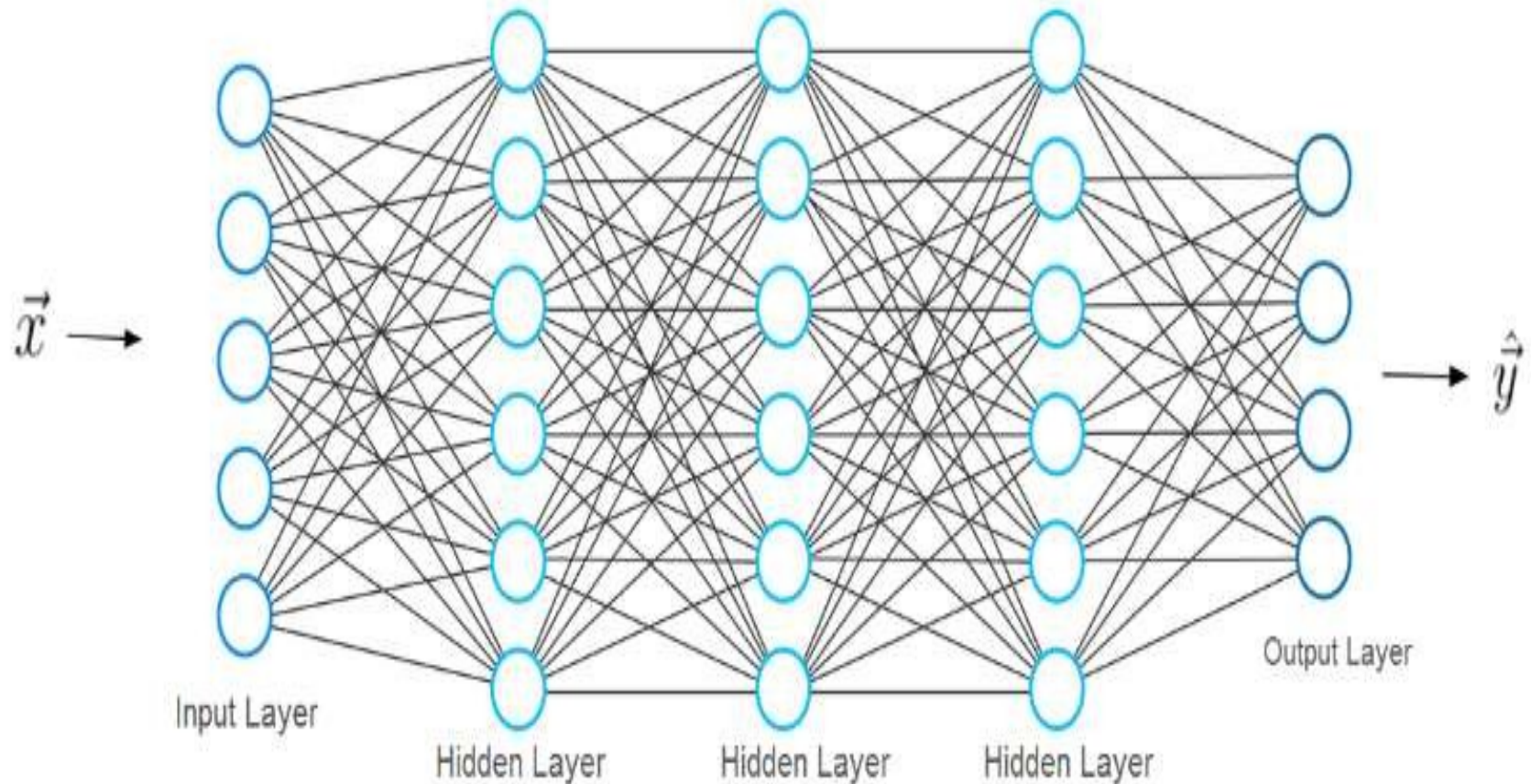


LAYERED CONNECTION OF NEURAL NETWORK

Three basic components are used in ANN
are

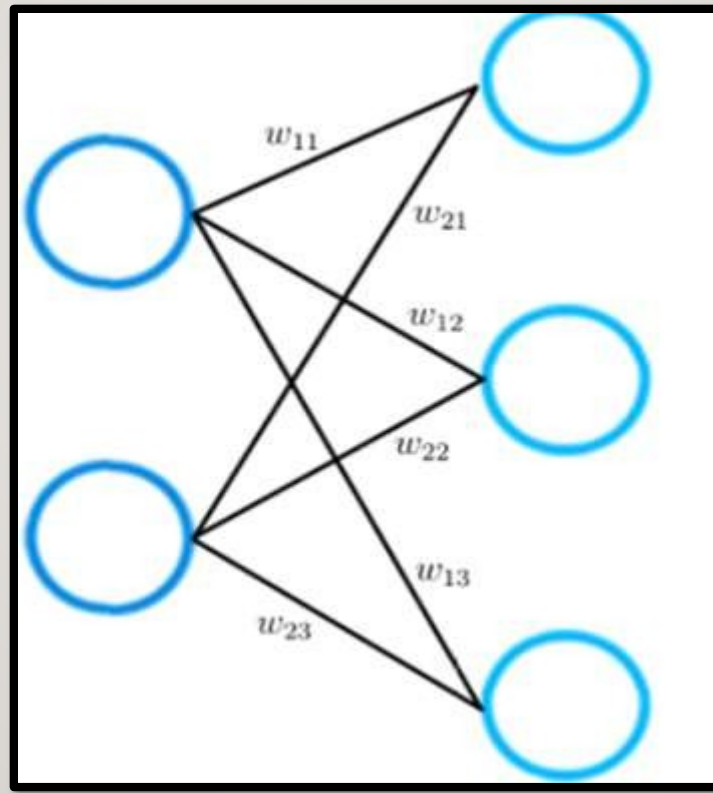
3. Activation Function

1. Components **2. Learning**



EXPLANATION OF LAYERS IN ANN

- The **input layer** has **two input neurons**, while the **output layer** consists of **three neurons**.



- **ACTIVATION FUNCTION:** It is also known as **Transfer Function**. It can also be attached with **2 neural networks**. It is also a **non linear Function**.
- Activation Functions are **Mathematical Functions** that determine the **output of a Neural Network**
- The **Activation Function** is attached to each neuron in a neural network and **determines whether the neuron should be activated or not**.



- The **main purpose** is to **convert** a **input signal of a node** in an ANN to an **Output Signal**.
- It translates the input signals to output signals. It maps the output values on a range like **0 to 1 or -1 to 1**.



PERCEPTRON

- ❑ **A Perceptron is an Artificial Neuron**
- ❑ **Frank Rosenblatt (1928 – 1971) was an American psychologist notable in the field of Artificial Intelligence.**
- ❑ **In 1957 , he "invented" a Perceptron program, on an IBM 704 computer at Cornell Aeronautical Laboratory.**
- ❑ **The Neurons receive input from our senses by electrical signals and store information to make decisions based on previous input. So, A Perceptron Unit is used to build the**

ANN System

Perceptron is a **Machine Learning algorithm** for supervised

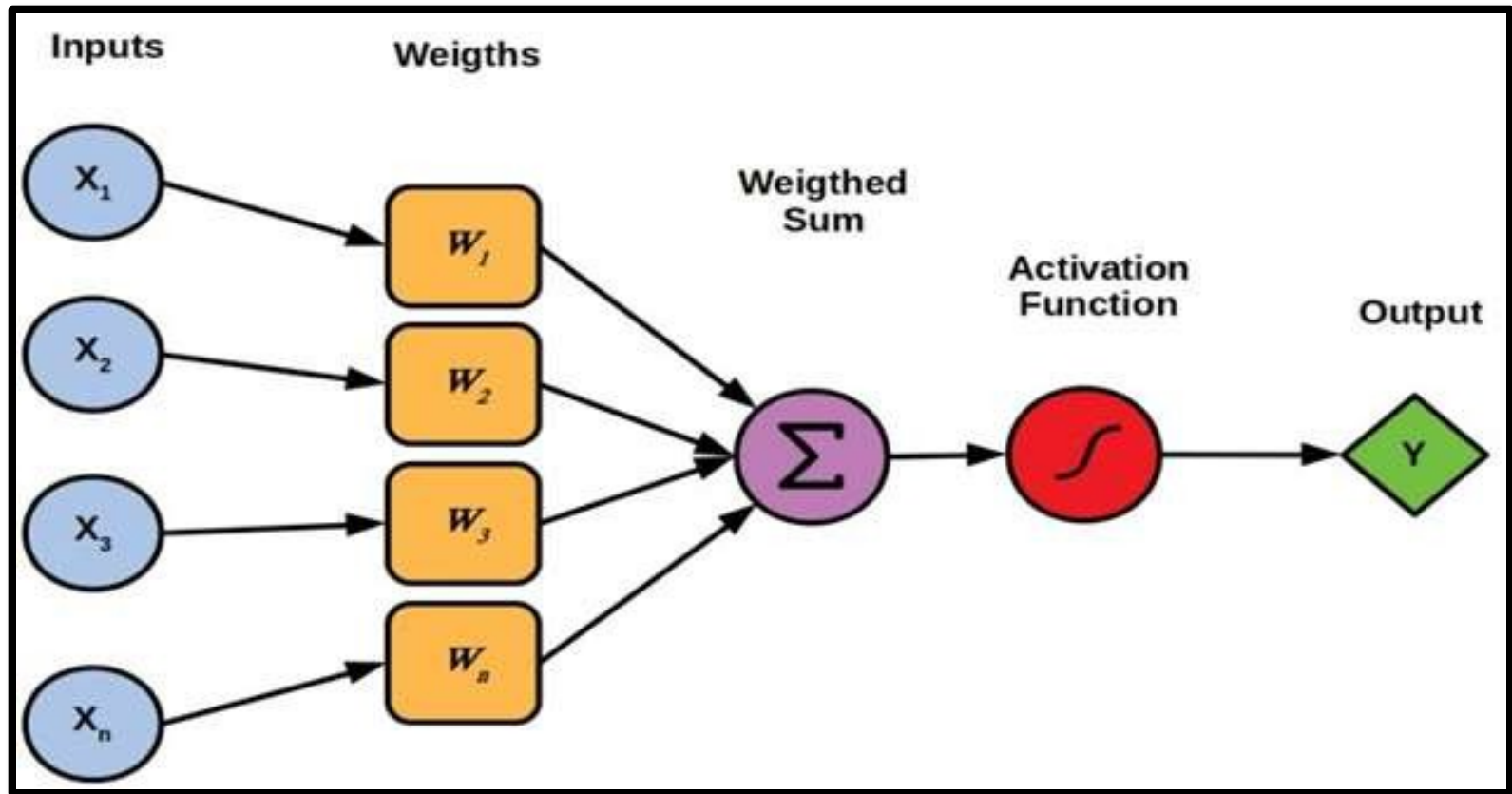
learning of various **binary classification tasks**.

The **original Perceptron** was designed to take a number of binary inputs, and **produce one binary output (0 or 1)**.

A perceptron is a fundamental building block of artificial intelligence and machine learning. Think of it as a simplified model of a neuron in our brain. It takes inputs, multiplies them by their respective weights, adds them up, and applies a threshold to determine its output.

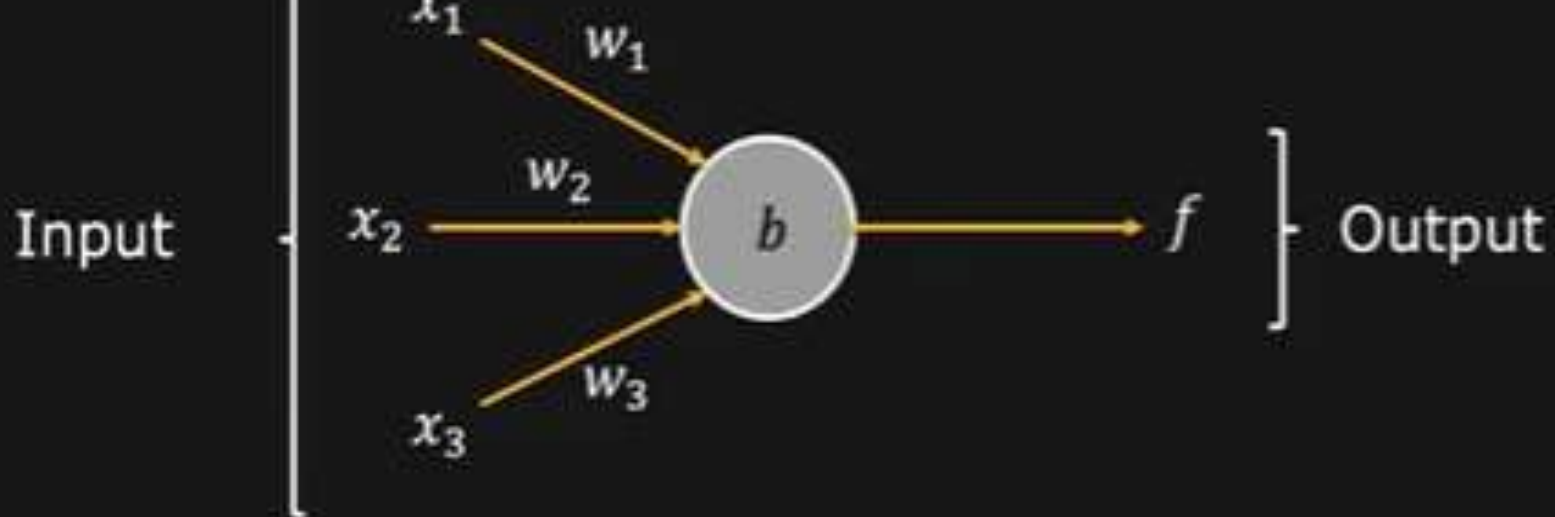
This output is then compared to the desired output, and the perceptron adjusts its weights using a process called gradient descent to minimize the difference between the actual and desired outputs.

□ Mr. Frank Rosenblatt invented the perceptron model as a binary classifier.



□ The **idea** was to **use different weights** to represent the importance of **each input**, and **that the sum of the values should be greater than a threshold value** before making a decision like true or false (0 or 1).





$$f = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq -b \\ 1 & \text{if } \sum_j w_j x_j > -b \end{cases}$$

$$f = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

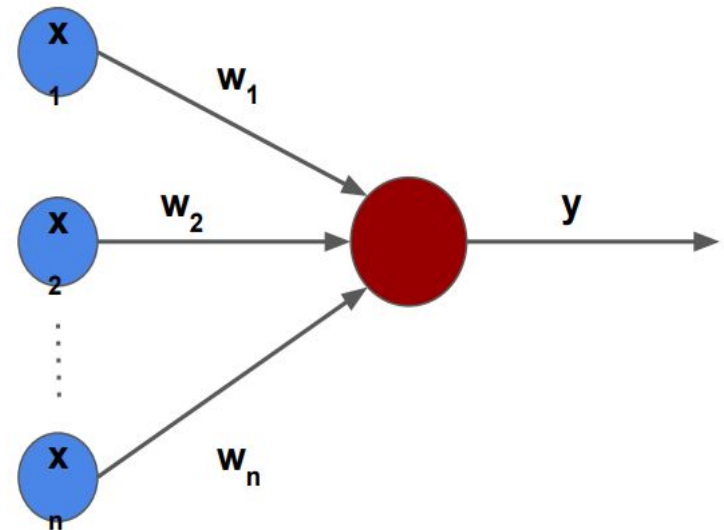
where, w_j = weights

b = bias (threshold)

The Perceptron Model

- Motivated by the biological neuron.
- A perceptron is a computing element where inputs are associated with the weights and the cell having a threshold value.

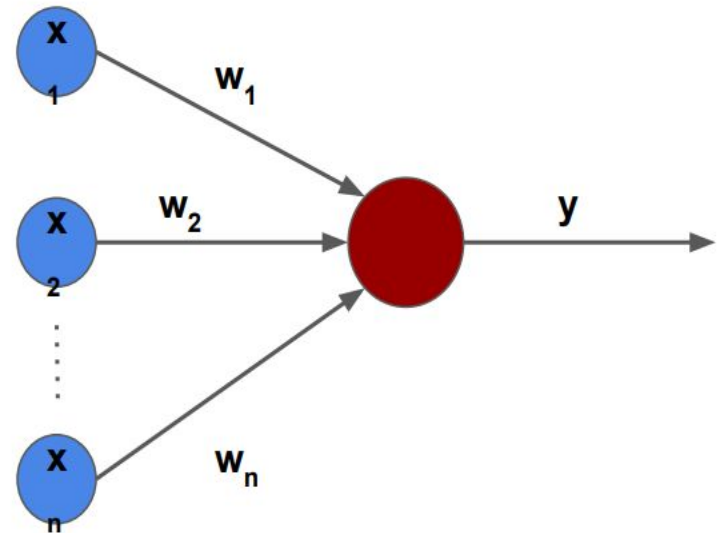
$$y = \begin{cases} 1, & \text{if } \sum w_i x_i > \text{threshold} \\ 0 & \text{otherwise} \end{cases}$$



The Perceptron Model

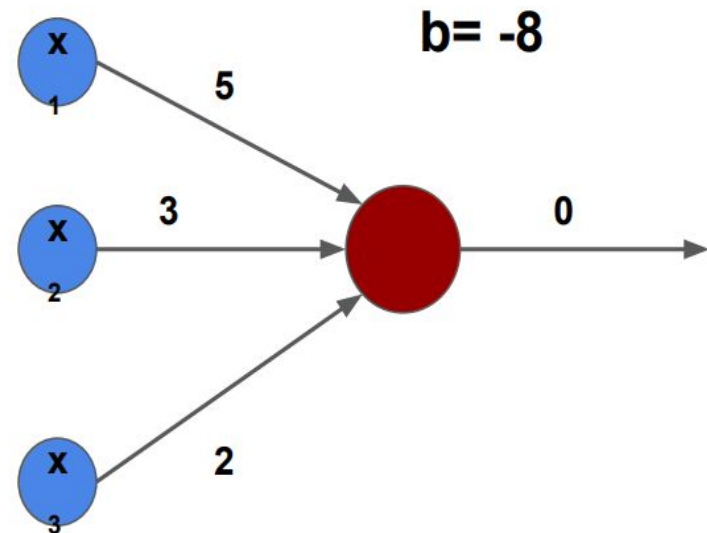
- Rewrite $\sum w_i x_i$ as $w.x$
- Replace threshold = -b
- **b**: Bias, a prior inclination towards some decision.

$$y = \begin{cases} 1, & \text{if } w.x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$



A simple decision via Perceptron

- *Whether should you go to watch movie this weekend?*
- The decision variables are:
 - Is there any extra lecture this weekend? ($x_1=1$)
 - Does your friend want to go with you? ($x_2=0$)
 - Do you have pending assignments due on the weekend? ($x_3=1$)



- More precisely, the **given inputs** x_1 through x_n , the **output** $o_1(x_1, \dots, x_n)$ computed by the perceptron is

$$o(x_1, \dots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n > 0 \\ -1 & \text{otherwise} \end{cases}$$

- The Perceptron receives multiple input signals, and if the sum of the input signals **exceeds a certain threshold**, it either **outputs a signal or does not return an output**.
- In the context of supervised learning and classification, this can then be used to **predict the class of a sample**.

PERCEPTRON LEARNING ALGORITHM

Frank Rosenblatt suggested this algorithm:

- Step-1: Set a threshold value**
- Step-2: Multiply all inputs with its weights**
- Step-3: Sum all the results**
- Step-4: Activate the output**

- Here, First, **multiply all input values with corresponding weight values and then add them** to determine the **weighted sum**. Mathematically, we can calculate the weighted sum as follows:

$$\sum w_i * x_i = x_1 * w_1 + x_2 * w_2 + \dots w_n * x_n$$

- and that **the sum of the values should be greater than a threshold value** before making a decision like true or false (0 or 1).
- Add a special term called **bias 'b'** to this weighted sum to **improve the model's performance**. $\sum w_i * x_i + b$

Activation Function of a Perceptron

Let: $z = w \cdot x + b$

$$\text{Activation: } a = f(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$$



Activation Function for a **Perceptron** is a **Step Function**.

- A simple activation function, such as a step function, can be used to explain Perceptron.
- The step function compares the weighted sum of the inputs to the threshold. If the sum is greater than or equal to the threshold, the activation function outputs 1. Otherwise, it outputs 0.



A perceptron is the simplest building block of **neural networks**, used to mimic the way neurons work in the human brain.

Because it's the simplest architecture within artificial neural networks, perceptrons are a great starting point for those unfamiliar with machine learning.

This algorithm performs binary classification tasks, dividing inputs into one of two categories. You can think of this type of algorithm as a “yes” or “no” decision-maker based on whether the weights of input values reach a specific activation value.



For example, imagine you are deciding whether to buy a coffee at your local coffee shop. Your “input variables” might be the temperature outside, your weekly budget, and how tired you feel. Each of these conditions has a level of importance or weight that influences your final decision.

A perceptron takes these inputs and combines their weights to see if they collectively reach a certain threshold. If they do, the perceptron would activate or buy the coffee in this scenario. If not, the perceptron would remain inactive, and you would decide not to buy coffee that day. Over time, the perceptron adjusts the weights of the input layers to improve decision-making.



EXAMPLE

- Imagine a perceptron (in your brain).
- The perceptron tries to decide if you should go to a concert.
- Is the artist good?
- Is the weather good?
- What weights should these facts have?

Criteria	Input	Weight
Artists is Good	$x_1 = 0 \text{ or } 1$	$w_1 = 0.7$
Weather is Good	$x_2 = 0 \text{ or } 1$	$w_2 = 0.6$
Friend will Come	$x_3 = 0 \text{ or } 1$	$w_3 = 0.5$
Food is Served	$x_4 = 0 \text{ or } 1$	$w_4 = 0.3$
Alcohol is Served	$x_5 = 0 \text{ or } 1$	$w_5 = 0.4$

1. Set a threshold value:

- Threshold = 1.5

2. Multiply all inputs with its weights:

- $x_1 * w_1 = 1 * 0.7 = 0.7$
- $x_2 * w_2 = 0 * 0.6 = 0$
- $x_3 * w_3 = 1 * 0.5 = 0.5$
- $x_4 * w_4 = 0 * 0.3 = 0$
- $x_5 * w_5 = 1 * 0.4 = 0.4$

3. Sum all the results:

- $0.7 + 0 + 0.5 + 0 + 0.4 = 1.6$ (The Weighted Sum)

4. Activate the Output:

- Return true if the sum > 1.5 ("Yes I will go to the Concert")

□ Perceptron Terminology:

□ 1. Inputs : Here Inputs called Nodes

In the above example, the node values are: **1, 0, 1, 0, 1** The binary **input values** (0 or 1) can be interpreted as (no or yes) or (false or true).

□ 2. Node Weights: Weights shows the **strength of each node.**

In the above example, the Weights are **0.7, 0.6, 0.5, 0.3, 0.4**



- **3. Activation Function:** The activation functions maps the result (the weighted sum) into a required value like 0 or 1.
- In the above Example, the activation function is simple: $(\text{sum} > 1.5)$
- The binary output (1 or 0) can be interpreted as (yes or no) or (true or false).
- **Note:** It is obvious that a *decision is NOT made by one neuron alone.*
- **Threshold Value:** We have to define a reliable threshold for the classifier that clearly indicates the split between the

- **Bias:** Model bias refers to the presence of **systematic errors** in a model **that can cause it to consistently make incorrect predictions.**
- These errors can arise from **many sources**, including the **selection of the training data, the choice of features used to build the model, or the algorithm used to train the model.**

REPRESENTATION OF POWER OF

PERCEPTRON

□ As we can see, it calculates a weighted sum of its inputs and thresholds it with a step function.

□ A single perceptron can be used to represent many

Boolean functions.

□ The Boolean Functions , AND, OR, XOR can be solved the Perceptron.

□ Geometrically, this means the perceptron can separate its input space with a Hyper Plane.

□ That's where the notion that a perceptron can **only separate linearly separable problems** .

□ Types of Perceptron's:

□ 1. Single Layer Perceptron Model

□ 2. Multi Layer Perceptron Model

□ 1. Single Layer Perceptron Model: The main objective of the single-layer perceptron model is **to analyze the linearly separable objects** with **binary outcomes**.

□ Perceptrons can represent all of the Boolean Functions , AND, OR ,NAND,NOR can be solved the Perceptron

□ Single Layer perceptron , can only solve Linearly

Separable Problems.

Single-Layer Perceptron Model

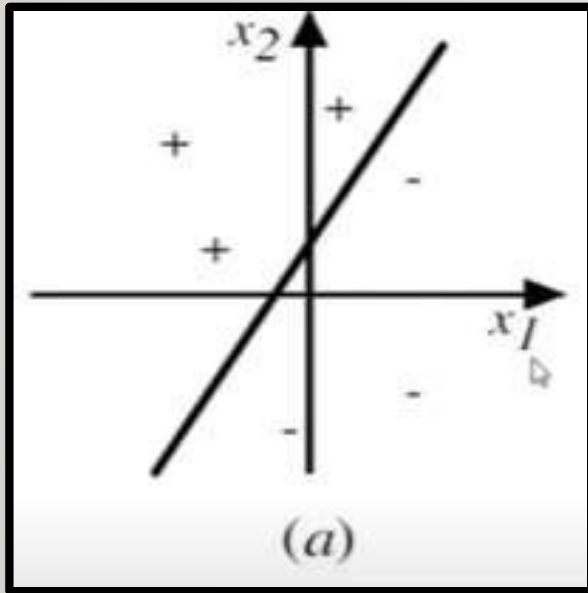
- The single-layer Perceptron is the most basic form of the Perceptron algorithm. It consists of only one layer of output neurons that are directly connected to input features.
- This model can only handle linearly separable data, meaning it can draw a straight line to classify data points into two categories.
- The mathematical formulation for the single-layer Perceptron involves computing the weighted sum of inputs and applying an activation function to determine the output. However, it cannot solve more complex problems that involve non-linear data, such as the XOR problem.



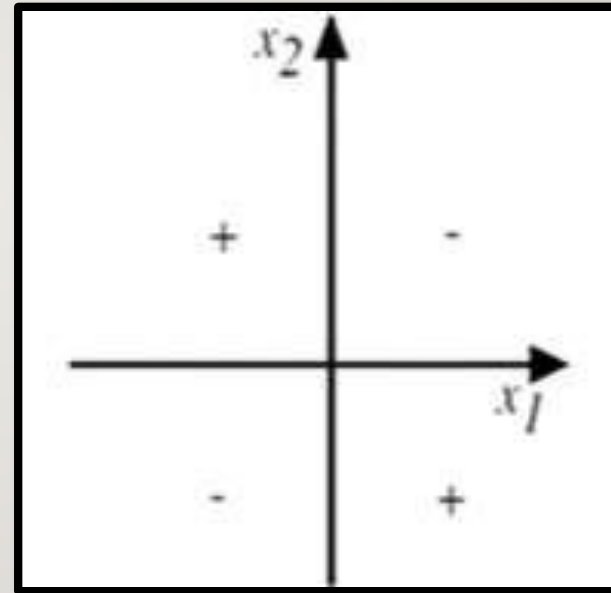
- **Multi-Layer Perceptron (MLP)**
- The Multi-Layer Perceptron (MLP) is an extension of the Perceptron model, introducing one or more hidden layers between the input and output layers. These hidden layers allow MLPs to handle non-linear data by using more complex activation functions like the sigmoid or ReLU (Rectified Linear Unit).
- The MLP employs backpropagation, a learning algorithm that adjusts weights across all layers to minimize prediction errors. MLPs are the foundation for modern deep learning models and are capable of solving more complex classification and regression tasks.



- Some of the **Boolean Functions** cannot represent the **Single Perceptron**, Such as **XOR function** is not linearly separable, it really is impossible for a **single hyper plane** to separate it.



Linearly Separable



Not Linearly Separable

- **2. Multi Layer Perceptron Model:** Like a single-layer perceptron model, a multi-layer perceptron model also has the same model structure but has a **greater number of hidden layers**.
- The multi-layer perceptron model is also known as the **Back Propagation Algorithm**.
- Hence, a **multi-layered perceptron model** has considered as **artificial neural networks** having **various layers** in which **activation function does not remain linear**, similar to a single layer perceptron model.

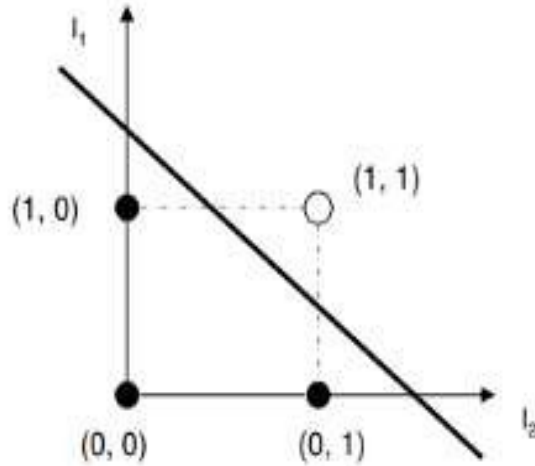
- ❑ **Instead of linear**, activation function can be executed as **sigmoid, TanH, ReLU, etc., for deployment.**
- ❑ A multi-layer perceptron model has **greater processing power** and can **process linear and non-linear patterns.**
- ❑ Further, it can also **implement logic gates** such as **AND, OR, XOR, NAND, NOT, XNOR, NOR.**

C
O
N
T
D
..

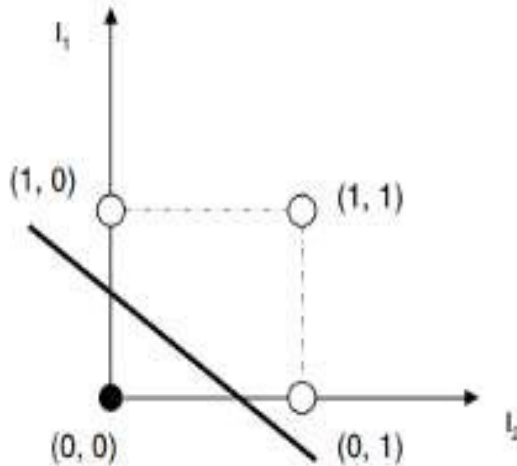


REPRESENTATION OF POWER OF PERCEPTRON

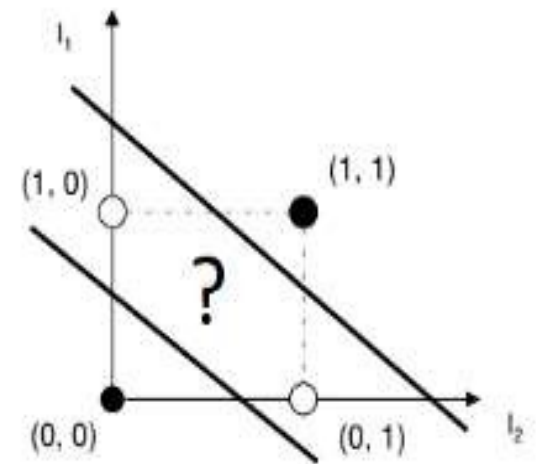
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



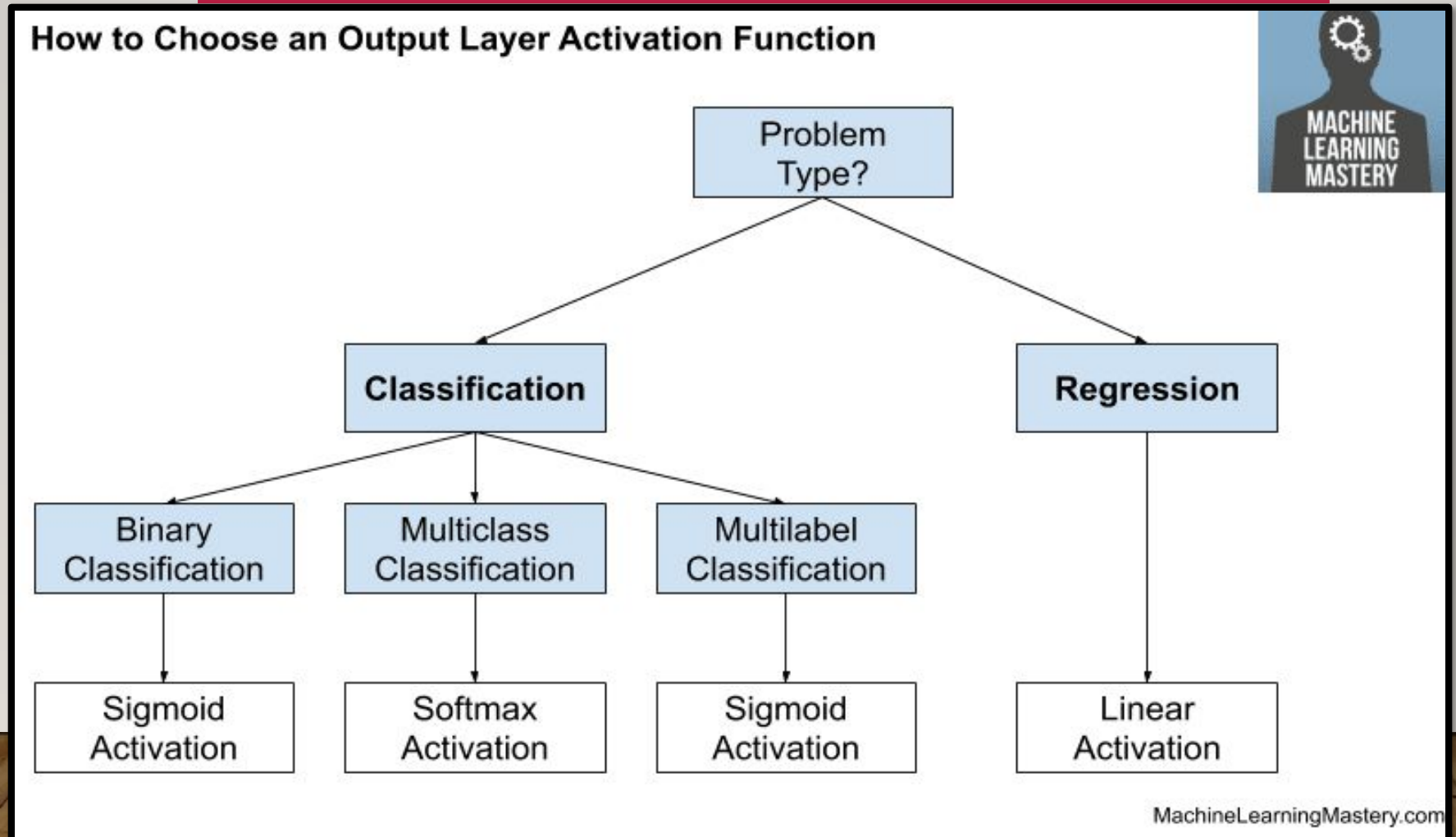
OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1



XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



□ Some of Different Activation Functions used in Machine Learning are:



Activation Functions (cont'd)

- **Sigmoid**

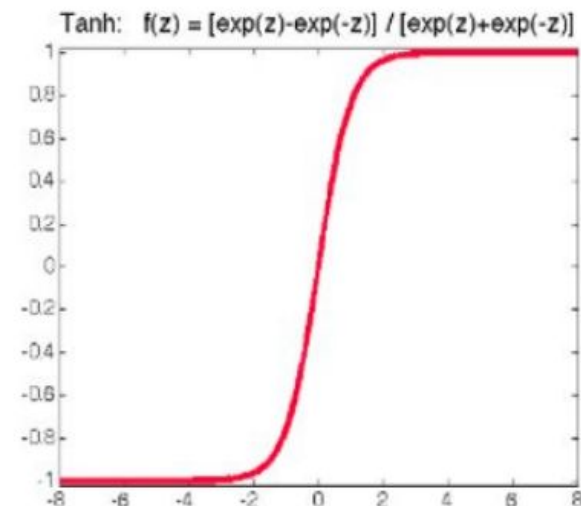
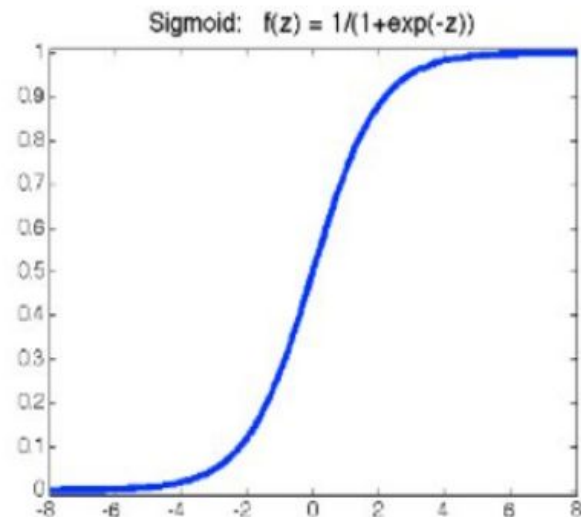
$$\sigma(z) = \frac{1}{1+\exp(-z)}$$

- continuously differentiable
- ranges from 0-1
- not symmetric around the origin

- **Tanh**

$$\tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$

- scaled version of the sigmoid
- symmetric around the origin
- vanishing gradient



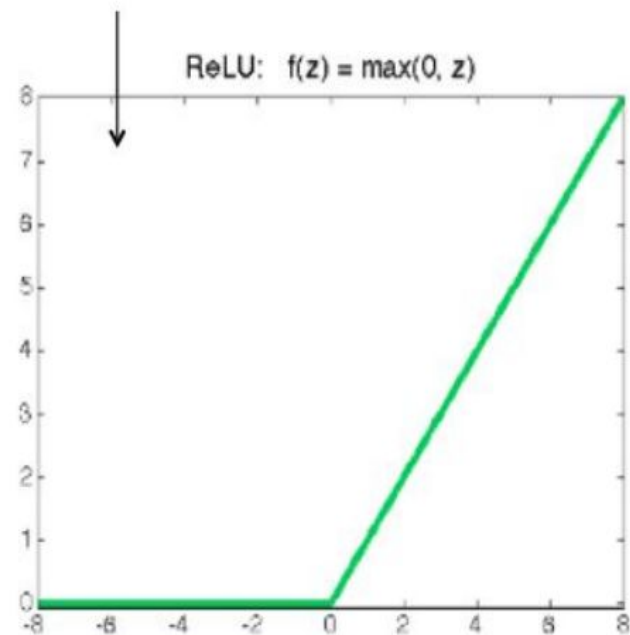
Activation Functions (cont'd)

- ReLU

$$\text{ReLU}(z) = \max(0, z)$$

- Also called piecewise linear function because rectified function is linear for half of the input domain and nonlinear for the other half.
- trivial to implement
- sparse representation
- avoid the problem of vanishing gradients
- **dead neurons**

Most popular recently
for deep learning



Importance of Loss Functions in Machine Learning

- Loss functions are integral to the training process of machine learning models. They provide a measure of how well the model's predictions align with the actual data. By minimizing this loss, models learn to make more accurate predictions.
- The choice of a loss function can significantly affect the performance of a model, making it crucial to select an appropriate one based on the specific task at hand.



Categories of Loss Functions

The loss function estimates how well a particular algorithm models the provided data. Loss functions are classified into two classes based on the type of learning task.

Regression Loss Functions in Machine Learning

Regression tasks involve predicting continuous values, such as house prices or temperatures. Here are some commonly used loss functions for regression:

1. Mean Squared Error (MSE)

It is the Mean of Square of Residuals for all the datapoints in the dataset. Residuals is the difference between the actual and the predicted prediction by the model.



This squaring process is essential for most regression loss functions, ensuring that models can minimize error and improve performance. The formula is:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where:

- y_i is the actual value for the i-th data point.
- $\hat{y}_i$ is the predicted value for the i-th data point.
- n is the total number of data points.

Student	Actual Marks (y)
A	80
B	65
C	90
D	75

Error ($y - \hat{y}$)	Squared Error	
2	4	
-5	25	
5	25	
2	4	

Step 1: Sum the squared errors

$$4 + 25 + 25 + 4 = 58$$

Step 2: Divide by the number of observations

$$\text{MSE} = \frac{58}{4} = 14.5$$

Answer:

$\text{MSE} = 14.5$

2. Mean Absolute Error (MAE)

The **Mean Absolute Error (MAE)** is a commonly used loss function in machine learning that calculates the mean of the absolute values of the residuals for all datapoints in the dataset.

- The absolute value of the residuals is taken to convert any negative differences into positive values, ensuring that all errors are treated equally.
- Taking the mean makes the loss function independent of the number of datapoints in the training set, allowing it to provide a consistent measure of error across datasets of different sizes.

The formula:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Where:

- y_i is the actual value for the i-th data point.
- $\hat{y}_i$ is the predicted value for the i-th data point.
- n is the total number of data points.

House	Actual Price (₹ Lakhs)	Predicted Price (₹ Lakhs)
1	50	48
2	60	63
3	55	54
4	70	68

Sum of absolute errors:

$$2 + 3 + 1 + 2 = 8$$

Number of houses:

$$4$$

MAE:

$$MAE = \frac{8}{4} = 2$$

Answer

$$\boxed{MAE = 2 \text{ lakhs}}$$

3. Mean Bias Error

It is similar to Mean Squared Error (MSE) but provides less accuracy. However, it can help in determining whether the model has a positive bias or negative bias. By analyzing the loss function results, you can assess whether the model consistently overestimates or underestimates the actual values. This insight allows for further refinement of the machine learning model to improve prediction accuracy. Such loss function examples are useful in understanding model performance and identifying areas for optimization, making them an essential part of the machine learning process.

The formula:

Formula

$$\text{MBE} = \frac{1}{n} \sum_{i=1}^n (P_i - A_i)$$

Where:

- P_i = Predicted value
- A_i = Actual (observed) value
- n = Number of observations

Interpretation

- **MBE > 0:** Model **overestimates** the actual values.
- **MBE < 0:** Model **underestimates** the actual values.
- **MBE = 0:** No average bias.

Actual	Predicted
100	105
120	125
130	135
150	155
200	205

Example 1: Positive Mean Bias Error (Overestimation)

Actual	Predicted	Error (Predicted - Actual)
100	105	+5
120	125	+5
130	135	+5
150	155	+5
200	205	+5

Step 1: Sum of errors

$$5 + 5 + 5 + 5 + 5 = 25$$

Step 2: Calculate MBE

$$\text{MBE} = \frac{25}{5} = 5$$

Interpretation:

The model **overestimates** the actual values by **5 units on average**.

Actual	Predicted
100	102
120	118
130	131
150	149

Actual	Predicted
100	98
120	118
130	127
150	147
200	198

MBE Value	Interpretation
Positive (+)	Model overestimates actual values
Negative (−)	Model underestimates actual values
Zero (0)	No average bias

- Classification Loss Functions in Machine Learning

1. Cross-Entropy Loss

- **Cross-Entropy Loss**, also known as Negative Log Likelihood, is a commonly used loss function in machine learning for classification tasks. This loss function measures how well the predicted probabilities match the actual labels.



- Cross-entropy loss measures the difference between the predicted probabilities and the actual class labels.
- The loss increases as the predicted probability moves farther away from the true class.
- Correct predictions with high confidence result in low loss.
- Incorrect predictions with high confidence result in high loss.
- During training, the model aims to minimize cross-entropy loss.
- Reducing the loss helps improve the accuracy and performance of classification models.
- In classification tasks, cross-entropy compares the predicted probability distribution with the true class label and updates the model to reduce prediction errors.

Formula

For **binary classification**:

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

Where:

- y = Actual class (0 or 1)
- p = Predicted probability of class 1

Example 1: Correct Prediction (Low Loss)

Suppose we are predicting whether an email is **Spam (1)** or **Not Spam (0)**.

- Actual class = **1**
- Predicted probability = **0.90**

Using the formula:

$$L = -(1 \log(0.90) + 0 \log(0.10))$$

$$L = -\log(0.90)$$

$$L = 0.105$$

Interpretation:

- Prediction is very close to the actual class.
- **Loss is very small (0.105).**

Example 2: Wrong Prediction (High Loss)

Actual class = 1

Predicted probability = 0.10

$$L = -\log(0.10)$$

$$L = 2.303$$

Interpretation:

- The model is very confident but wrong.
- **Loss is very high (2.303).**

Multi-Class Cross Entropy

For multiple classes:

$$L = - \sum_{i=1}^C y_i \log(p_i)$$

Where:

- C = Number of classes
- y_i = One-hot encoded actual class
- p_i = Predicted probability

Example 4: Three-Class Classification

Suppose a model classifies an image into:

- Cat
- Dog
- Horse

Actual Label

Dog

One-hot encoding:

[0, 1, 0]

Predicted Probabilities

Class	Probability
Cat	0.10
Dog	0.80
Horse	0.10

Cross Entropy Loss

$$L = -(0 \log 0.10 + 1 \log 0.80 + 0 \log 0.10)$$

$$L = -\log 0.80$$

$$L = 0.223$$

The prediction is good, so the loss is low.

Key Points

- **Lower cross entropy loss** indicates better predictions.
- **Loss = 0** means the model predicts the correct class with 100% probability.
- Cross entropy is widely used with **Softmax** (multi-class classification) and **Sigmoid** (binary classification) in neural networks.



2. Hinge Loss

Hinge Loss, also known as Multi-class SVM Loss, is a type of loss function used for maximum-margin classification tasks, most commonly applied in **support vector machines (SVMs)**. This loss function in machine learning is particularly effective in ensuring that the decision boundary is as far away as possible from any data points. Hinge Loss is a convex function, making it suitable for optimization using a convex optimizer.



This type of loss function is widely used in classification tasks as it encourages models to achieve a larger margin between different classes, leading to better generalization. A common loss function example involving Hinge Loss can be seen in SVM models.

The formula:

Formula

$$\text{Hinge Loss} = \max(0, 1 - y\hat{y})$$

Where:

- y = True label (+1 or -1)
- $\hat{y}$ = Predicted score (not a probability)

Example 1: Correct Classification with Large Margin

- True label (y) = +1
- Predicted score ($\hat{y}$) = 2

Calculation:

$$\begin{aligned}\text{Loss} &= \max(0, 1 - (1 \times 2)) \\ &= \max(0, -1) = 0\end{aligned}$$

Interpretation:

The prediction is correct and exceeds the required margin, so the **loss is 0**.

Example 2: Correct Classification but Small Margin

- True label = +1
- Predicted score = 0.7

Calculation:

$$\begin{aligned}\text{Loss} &= \max(0, 1 - (1 \times 0.7)) \\ &= \max(0, 0.3) = 0.3\end{aligned}$$

Interpretation:

The prediction is correct but does not meet the required margin, so the model receives a **small penalty**.

Example 3: Incorrect Classification

- True label = +1
- Predicted score = -0.5

Calculation:

$$\begin{aligned}\text{Loss} &= \max(0, 1 - (-0.5)) \\ &= \max(0, 1.5) = 1.5\end{aligned}$$

Interpretation:

The prediction is incorrect, resulting in a **high loss**.

Key Points

- **Loss = 0**: Correct classification with a margin of at least 1.
- **$0 < \text{Loss} < 1$** : Correct classification, but the margin is insufficient.
- **Loss > 1**: Incorrect classification.
- Hinge loss is mainly used in **Support Vector Machines (SVMs)** to maximize the separation (margin) between classes.

3. Kullback-Leibler Divergence (KL Divergence)

Kullback-Leibler (KL) Divergence, also called Relative Entropy, is a metric that measures how one probability distribution differs from another probability distribution.

Definition

KL Divergence measures the amount of information lost when an estimated (predicted) probability distribution Q is used instead of the true probability distribution P .

Formula

$$D_{KL}(P||Q) = \sum_i P(i) \log \left(\frac{P(i)}{Q(i)} \right)$$

Where:

- $P(i)$ = True probability distribution
- $Q(i)$ = Predicted/Estimated probability distribution
- D_{KL} = KL Divergence

Example 1

Suppose the true distribution is

Class	True Probability (P)
-------	----------------------

Cat	0.6
-----	-----

Dog	0.4
-----	-----

Predicted distribution

Class	Predicted Probability (Q)
-------	---------------------------

Cat	0.5
-----	-----

Dog	0.5
-----	-----

Step 1

For Cat

$$0.6 \log \left(\frac{0.6}{0.5} \right)$$

$$= 0.6 \times 0.182$$

$$= 0.109$$

Step 2

For Dog

$$0.4 \log \left(\frac{0.4}{0.5} \right)$$

$$= 0.4 \times (-0.223)$$

$$= -0.089$$

Step 2

For Dog

$$\begin{aligned} & 0.4 \log \left(\frac{0.4}{0.5} \right) \\ &= 0.4 \times (-0.223) \\ &= -0.089 \end{aligned}$$

Step 3

Total KL Divergence

$$0.109 - 0.089 = 0.020$$

Interpretation

Since the value is **close to zero**, the predicted distribution is very similar to the true distribution.

Example 2

Class	P
A	0.8
B	0.2

Predicted Distribution

Class	Q
A	0.4
B	0.6

Key Points

- KL Divergence measures the **difference between two probability distributions**.
- **KL = 0** means the distributions are identical.
- **Smaller KL values** indicate better predictions.
- **Larger KL values** indicate greater differences between the predicted and true distributions.

Choosing the Right Loss Function

The choice of a loss function in machine learning is influenced by several key factors:

1. **Nature of the Task:** Determine whether you are dealing with regression or classification problems.
2. **Presence of Outliers:** Consider how outliers in your dataset may impact your decision; some loss functions (e.g., Mean Absolute Error (MAE) and Huber loss) are more robust to outliers than others.
3. **Model Complexity:** Simpler models may benefit from more straightforward loss functions, such as Mean Squared Error (MSE) or Cross-Entropy.
4. **Interpretability:** Some loss functions provide more intuitive explanations than others, making them easier to understand in

Loss Function	Used For	Formula	Output Range	Advantages	Disadvantages
Mean Squared Error (MSE)	Regression	$(\frac{1}{n} \sum (y - \hat{y})^2)$	0 to ∞	Penalizes large errors, easy to optimize	Sensitive to outliers
Mean Absolute Error (MAE)	Regression	$(\frac{1}{n} \sum y - \hat{y})$	0 to ∞		
Mean Bias Error (MBE)	Regression	$(\frac{1}{n} \sum (\hat{y} - y))$	Negative or Positive	Shows whether predictions are over- or under-estimated	Positive and negative errors can cancel out
Cross-Entropy Loss	Binary & Multi-class Classification	$(-\sum y \log(\hat{y}))$	0 to ∞	Excellent for probability-based classification	Large penalty for confident wrong predictions
Hinge Loss	Support Vector Machine (SVM)	$(\max(0, 1 - y\hat{y}))$	0 to ∞	Maximizes the margin between classes	Requires labels as +1 and -1
KL Divergence	Comparing Probability Distributions	$(\sum P \log \frac{P}{Q})$	0 to ∞	Measures difference between distributions	Not symmetric (($KL(P Q) \neq KL(Q P)$))

Key Differences

Loss Function	Regression / Classification	Main Purpose
MSE	Regression	Minimize squared prediction errors
MAE	Regression	Minimize absolute prediction errors
MBE	Regression	Measure prediction bias (overestimation or underestimation)
Cross-Entropy	Classification	Compare predicted probabilities with true labels
Hinge Loss	Classification (SVM)	Maximize the margin between classes
KL Divergence	Probability Models	Measure the difference between two probability distributions

Training Neural Networks

Training a neural network means teaching the network to make accurate predictions by adjusting its weights and biases based on the error in its predictions.

The main objective is:

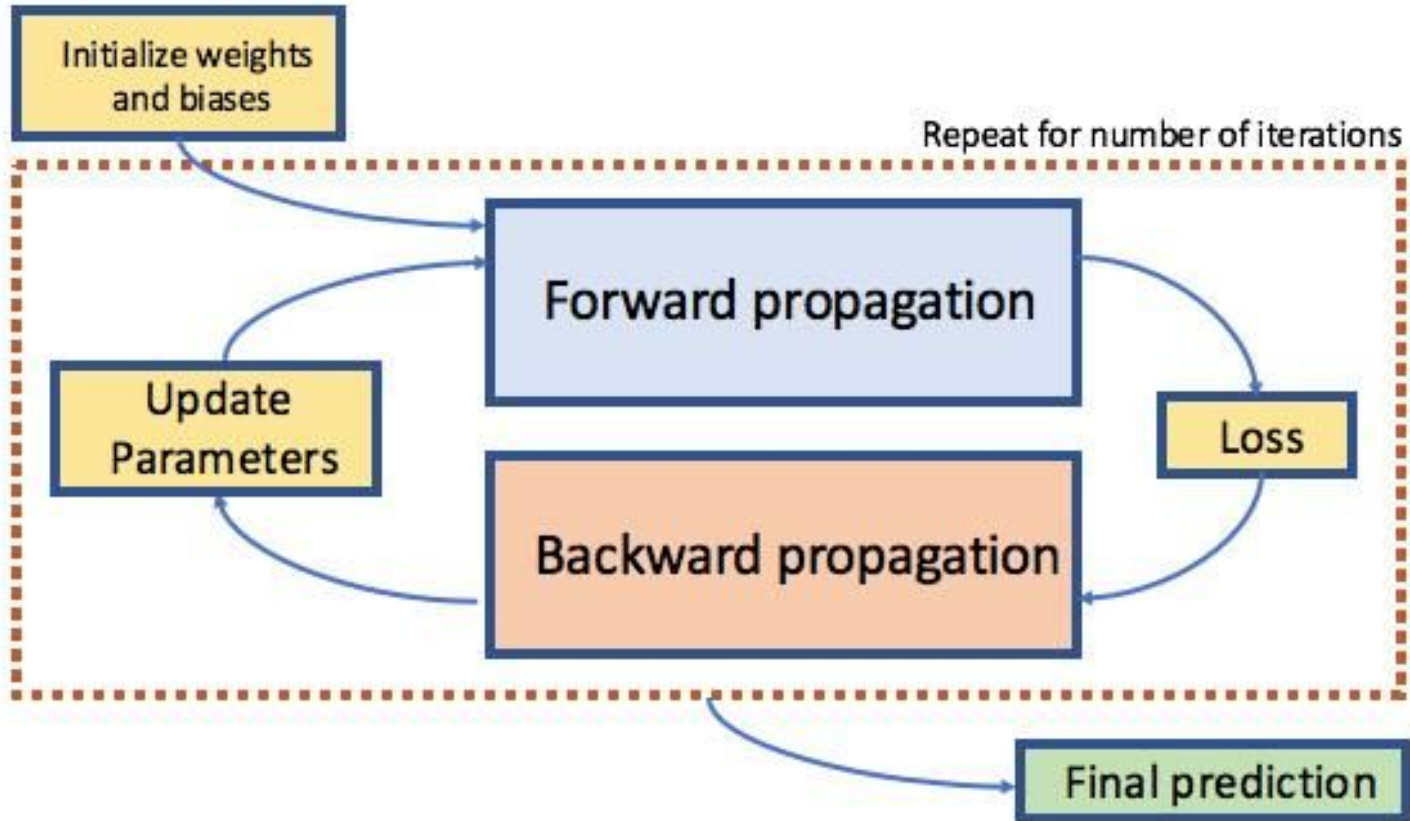
Minimize the difference between the predicted output and the actual output.

The training process can be understood as:

**Input Data → Forward Propagation → Prediction → Loss Calculation
→ Backpropagation → Weight Update → Repeat**

Training a neural network mainly involves two important processes:

1. Forward Propagation – Used to calculate the prediction.
2. Backpropagation – Used to calculate the error and update the weights.



Step 1: Initialize Weights

Randomly initialize weights and biases.

Step 2: Forward Propagation

Pass input through the network and calculate predicted output.

Step 3: Calculate Loss

Compare predicted output with actual output.

Step 4: Backpropagation

Calculate gradients by propagating the error backward.

Step 5: Update Weights

Use gradient descent to update weights.

Step 6: Repeat

Repeat the process for many iterations or epochs until the loss becomes sufficiently small.

1. Forward Propagation

Forward propagation is the process of passing input data from the input layer → hidden layer(s) → output layer to generate a prediction.

Step 1: Weighted Sum

Each neuron calculates:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

or,

$$z = \sum_{i=1}^n w_i x_i + b$$

where:

- x = input
- w = weight
- b = bias
- z = weighted sum

Step 2: Apply Activation Function

The neuron applies an activation function:

$$a = f(z)$$

For example, using the **Sigmoid function**:

$$a = \frac{1}{1 + e^{-z}}$$

The output a is then passed to the next layer.

Simple Example of Forward Propagation

Suppose:

$$x_1 = 2, \quad x_2 = 3$$

Weights:

$$w_1 = 0.4, \quad w_2 = 0.5$$

Bias:

$$b = 0.1$$

Calculate weighted sum:

$$z = (0.4)(2) + (0.5)(3) + 0.1$$

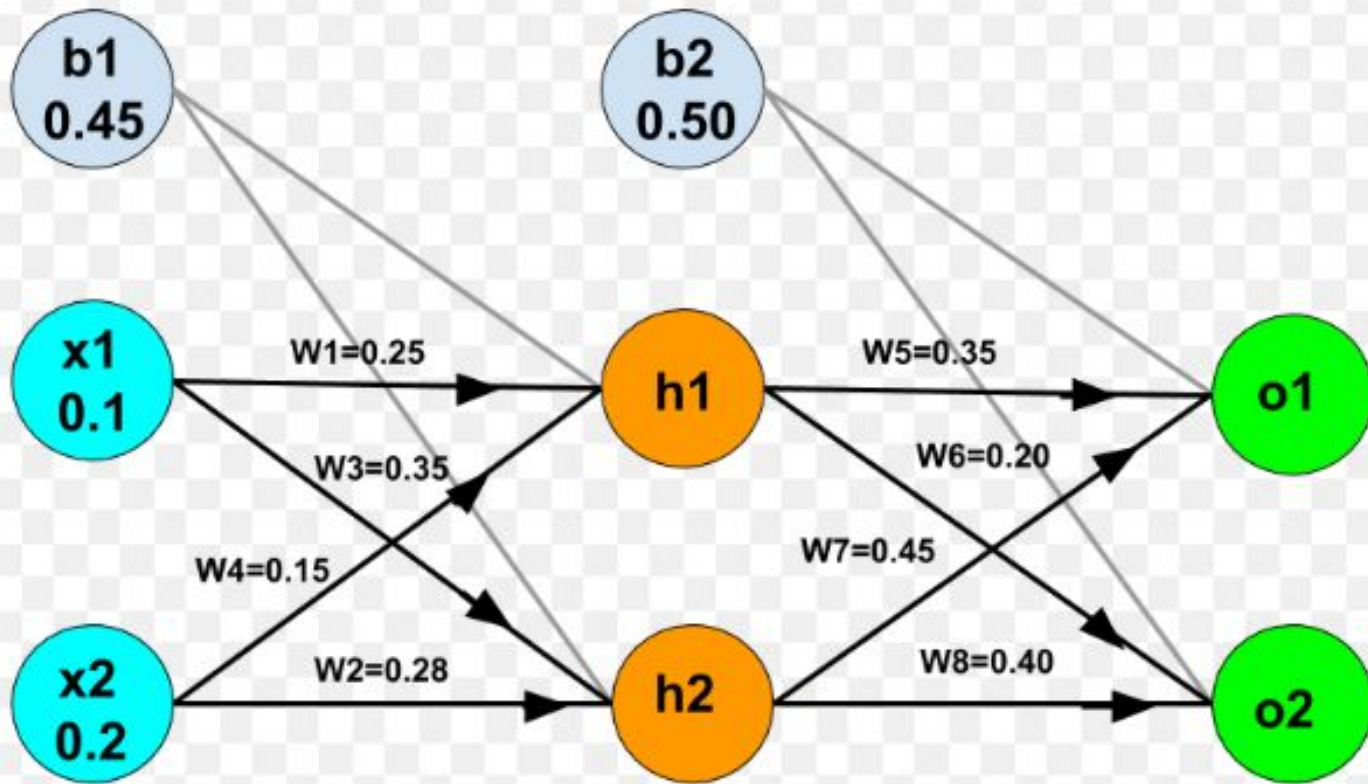
$$z = 0.8 + 1.5 + 0.1 = 2.4$$

Apply sigmoid:

$$a = \frac{1}{1 + e^{-2.4}}$$

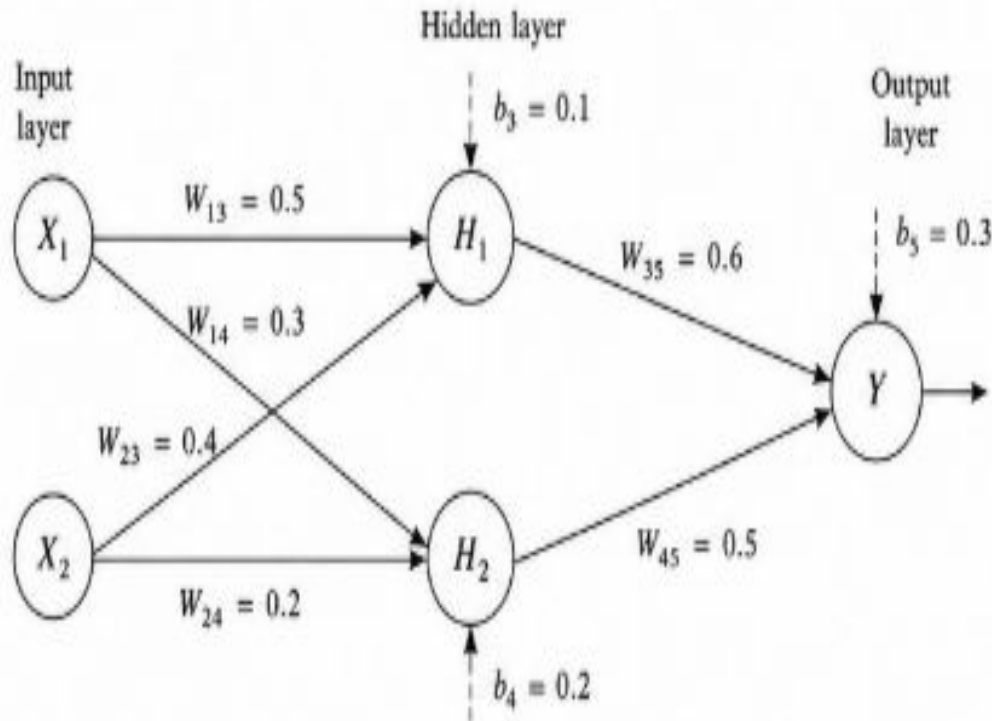
$$a \approx 0.9168$$

So, the **neuron's output** = 0.9168.



Q. Consider the feed-forward neural network shown below.

The network has two input neurons (X_1, X_2), two hidden neurons (H_1, H_2) and one output neuron (Y). Sigmoid activation function is used for all neurons.



Initial Weights and Biases

(a) Weights

$$W_{13} = 0.5 \quad W_{14} = 0.3$$

$$W_{23} = 0.4 \quad W_{24} = 0.2$$

$$W_{35} = 0.6 \quad W_{45} = 0.5$$

(b) Biases

$$b_3 = 0.1 \quad b_4 = 0.2 \quad b_5 = 0.3$$

For the given training tuple, perform one iteration of forward propagation and backpropagation.

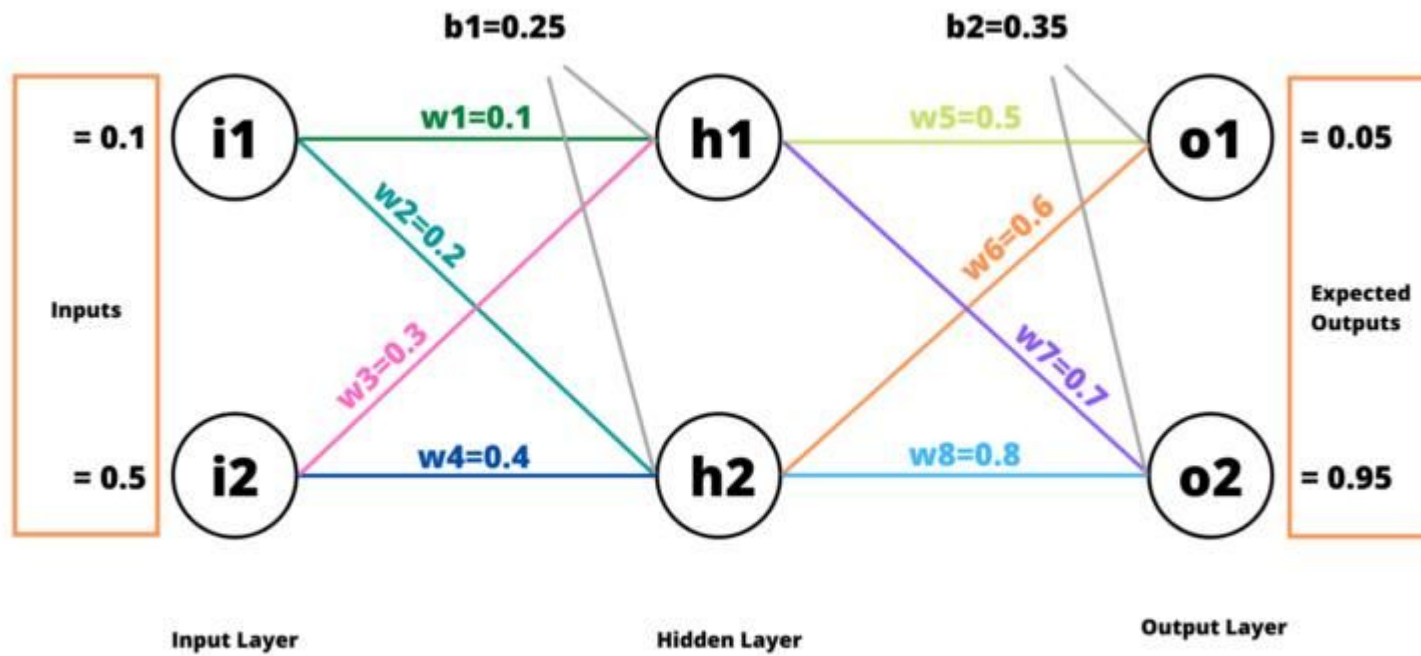
Training tuple

$$(X_1, X_2, t) = (1, 1, 1)$$

where $t = 1$ is the target output.

Learning rate:

$$\eta = 0.5$$



HYPERPARAMETERS: LEARNING RATE, REGULARIZATION, MOMENTUM, SPARSITY, GRADIENT DESCENT, VANISHING AND EXPLODING GRADIENT.

LEARNING RATE- The learning rate is a key hyperparameter that controls how quickly a model learns by determining the step size during weight updates.

- Controls how much weights are updated in response to error
- Determines step size while minimizing the loss function
- Affects speed and stability of training
- Too high may overshoot minimum; too low may slow learning



Where:

$$w = w - \alpha \cdot \nabla L(w)$$

- w represents the weights
- α
- α is the learning rate
- $\nabla L(w)$
- $\nabla L(w)$ is the gradient of the loss function

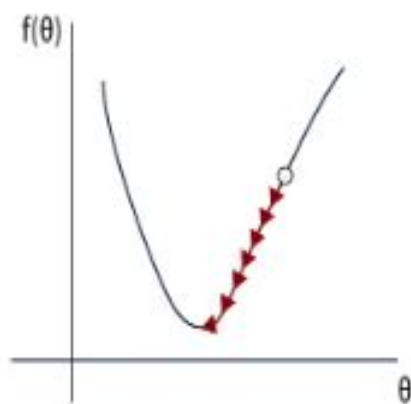
Impact of Learning Rate on Model

The learning rate directly influences how fast and how well a model learns by controlling the size of weight updates during training.

- A low learning rate leads to slow convergence, requires more epochs and increases computation time but can improve accuracy
- A high learning rate speeds up training but may overshoot optimal values and cause instability or divergence
- An optimal learning rate balances speed and accuracy, ensuring stable convergence
- Fine-tuning the learning rate is important for better performance
- Techniques like **learning rate scheduling** and adaptive optimizers help improve stability and efficiency

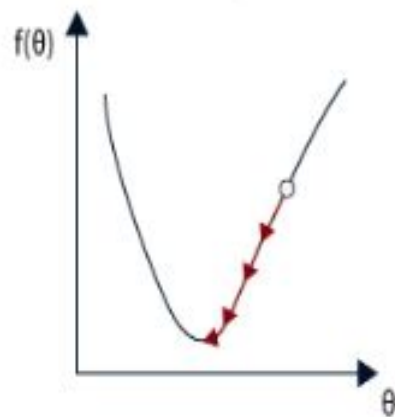


Too low



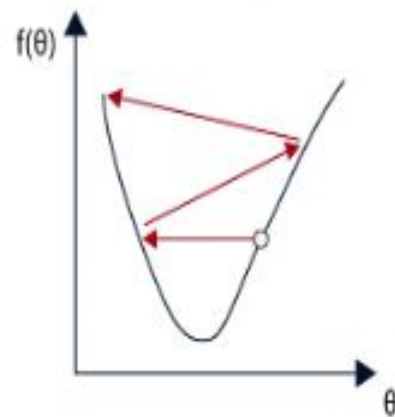
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point.

Too high



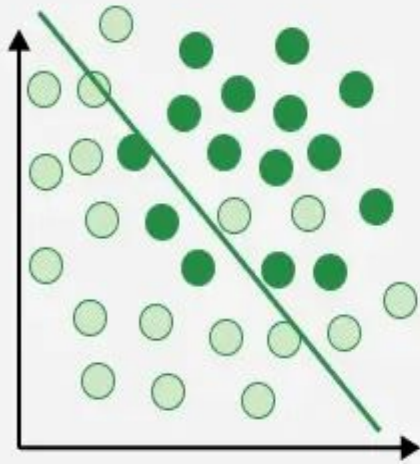
Too large of a learning rate causes drastic updates which lead to divergent behaviors.

REGULARIZATION

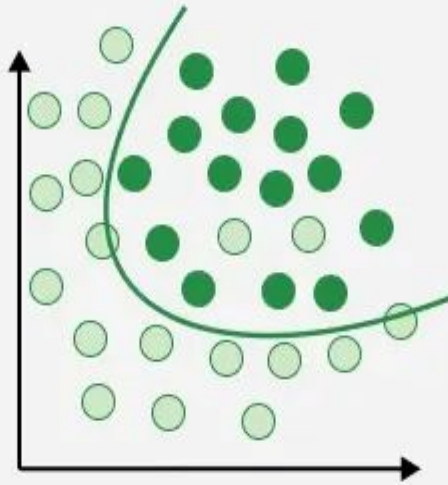
Regularization is a technique used in machine learning to prevent overfitting, which otherwise causes models to perform poorly on unseen data. By adding a penalty for complexity, regularization encourages simpler and more generalizable models.

- Prevents overfitting: Adds constraints to the model to reduce the risk of memorizing noise in the training data.
- Improves generalization: Encourages simpler models that perform better on new, unseen data.
- The main types include L1 (Lasso) and L2 (Ridge) regularization.

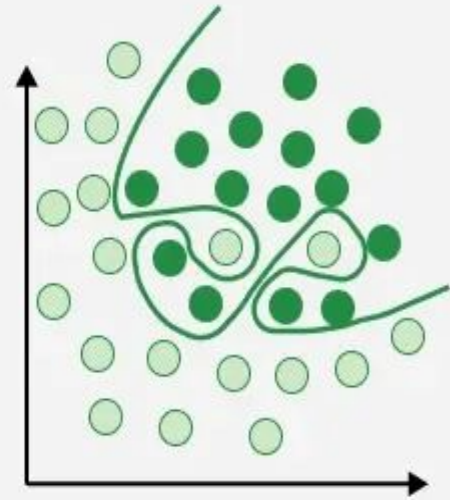




Under-fitting



Appropriate-fitting



Over-fitting

1. Lasso Regression

A regression model which uses the L1 Regularization technique is called **LASSO (Least Absolute Shrinkage and Selection Operator)** regression. It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function(L). This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.

2. Ridge Regression

A regression model that uses the L2 regularization technique is called **Ridge regression**. It adds the squared magnitude of the coefficient as a penalty term to the loss function(L). It handles multicollinearity by shrinking the coefficients of correlated features, reducing their variance and preventing any single feature from dominating the model.

3. Elastic Net Regression

Elastic Net Regression is a combination of both L1 as well as L2 regularization. It combines both L1 (absolute values) and L2 (squared values) penalties on the coefficients. With the help of an extra hyperparameter that controls the ratio of the L1 and L2 regularization.



MOMENTUM-

- Optimization algorithms are frequently used in machine learning models to identify the best collection of parameters that minimize a particular cost function. Momentum is a common optimization technique that is frequently utilized in machine learning.
- Momentum is a strategy for accelerating the convergence of the optimization process by including a momentum element in the update rule.
- This momentum factor assists the optimizer in continuing to go in the same direction even if the gradient changes direction or becomes zero. This can aid in improving convergence speed, reducing oscillations, avoiding becoming trapped in local minima, and making the optimization process more resistant to noisy gradients.

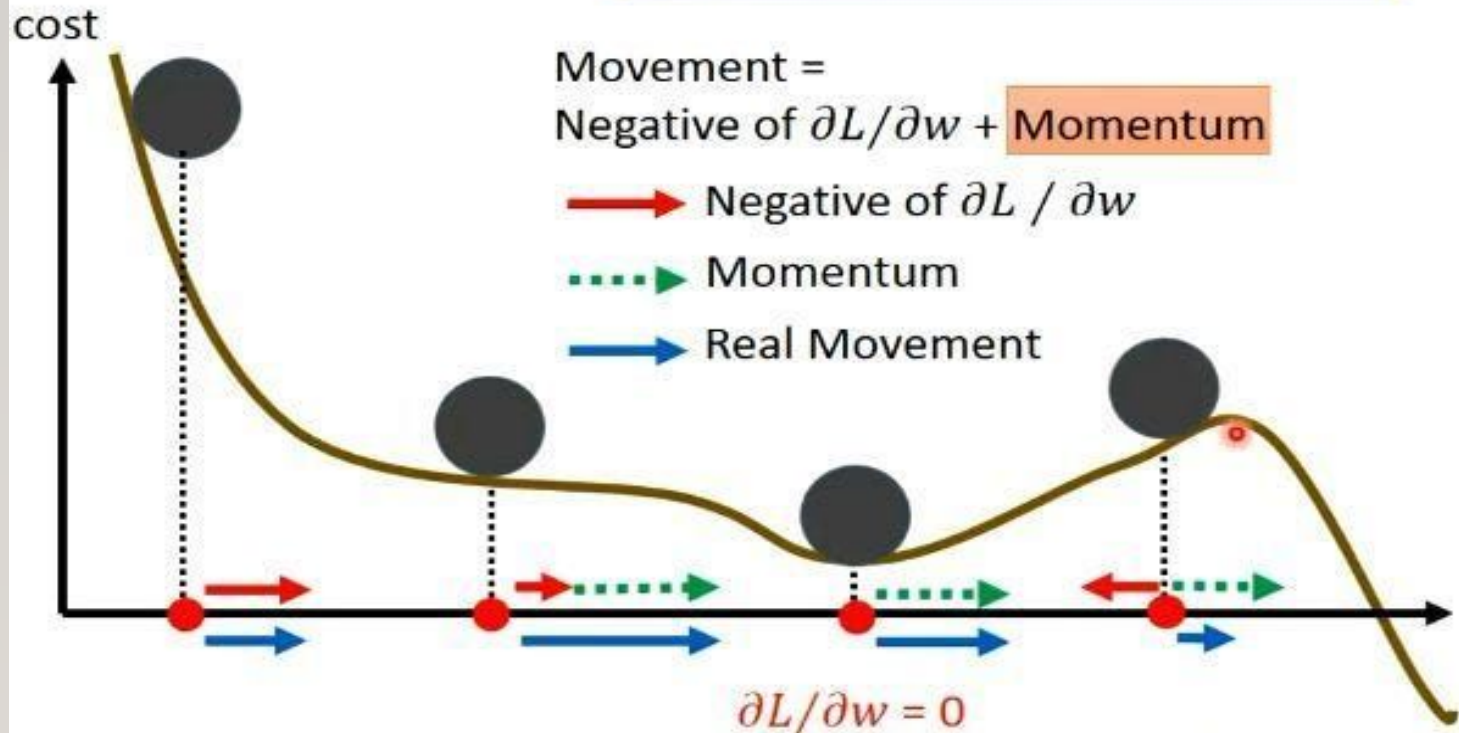


- Momentum is a gradient descent optimization technique that speeds up the learning process.
- It adds a fraction of the previous update to the current parameter update.
- This helps smooth parameter updates and reduces oscillations during training.
- Momentum uses an exponentially weighted moving average of past gradients as a form of memory.
- It helps the optimizer maintain its previous direction, leading to faster and more stable convergence.



Momentum

Still not guarantee reaching global minima, but give some hope

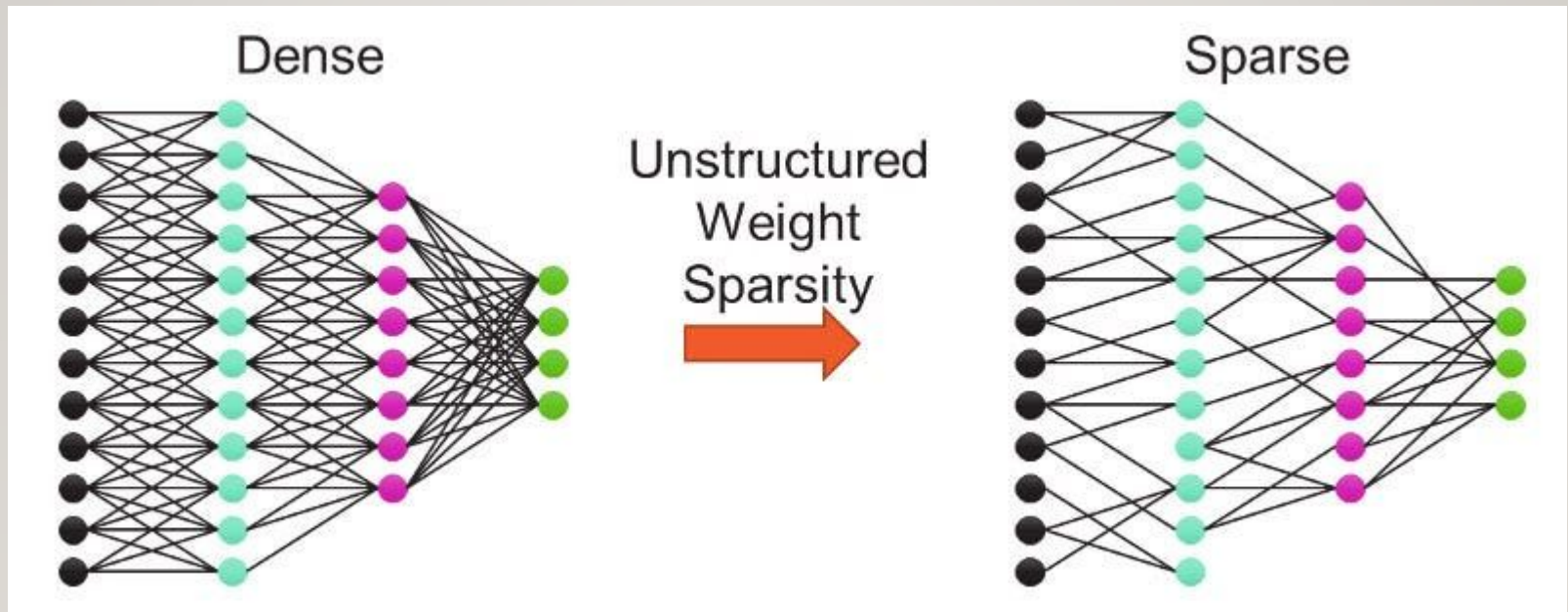


SPARSITY

- Sparse Representation in Deep Learning is a technique where most values in a data representation are zero, and only a few important values are non-zero. It helps deep learning models become more efficient and focus on the most relevant features.
- Reduces computation and memory usage
- Helps identify important patterns and features in data Can be applied to both model weights and hidden layer activations
- In NLP, word embeddings often result in sparse vectors since only a few features may be non-zero for a given word.
- In computer vision, an image might only have sparse features like edges or textures in specific areas.



- Using sparse representations helps deep learning models focus on relevant features, reducing the noise and improving their performance.



Gradient Descent is an iterative optimization algorithm used to minimize a model's cost function by strategically adjusting its internal parameters (weights and biases).

- Think of it as navigating down a dark mountain to find the lowest valley; you feel the slope under your feet and take small steps in the steepest downhill direction.
- In machine learning, the "mountain" represents the model's prediction error, and the "lowest valley" represents the global minimum where errors are minimized.



Core Components

- Cost Function: Measures how inaccurate a model's predictions are compared to real targets.
- Gradient: The vector of partial derivatives indicating the direction of steepest error increase.
- Learning Rate (α): A tuning hyperparameter determining the size of each step downward.



Working of Gradient Descent

- Step 1 we first initialize the parameters of the model randomly
- Step 2 Compute the gradient of the cost function with respect to each parameter. It involves making partial differentiation of cost function with respect to the parameters.
- Step 3 Update the parameters of the model by taking steps in the opposite direction of the model. Here we choose a **hyperparameter learning rate** which is denoted by γ . It helps in deciding the step size of the gradient.
- Step 4 Repeat steps 2 and 3 iteratively to get the best parameter for the defined model.

VANISHING AND EXPLODING GRADIENT

- Vanishing and exploding gradients are training instabilities that occur in deep neural networks when learning signals become either too small or too large.
- During backpropagation, gradients are calculated using the mathematical chain rule, which continuously multiplies partial derivatives layer by layer from the output back to the input.
- If these multiplied values are consistently less than one, the final gradient shrinks exponentially (vanishes); if they are greater than one, the gradient grows exponentially (explodes) .



- **The Vanishing Gradient Mechanics**
- When using traditional activation functions like Sigmoid or Tanh, the maximum value of their derivatives is small (0.25 for Sigmoid). Multiplying these fractions repeatedly across twenty or more hidden layers causes the gradient to decay to near-zero before reaching the initial layers. As a result, the early layers fail to learn crucial low-level features from raw data.

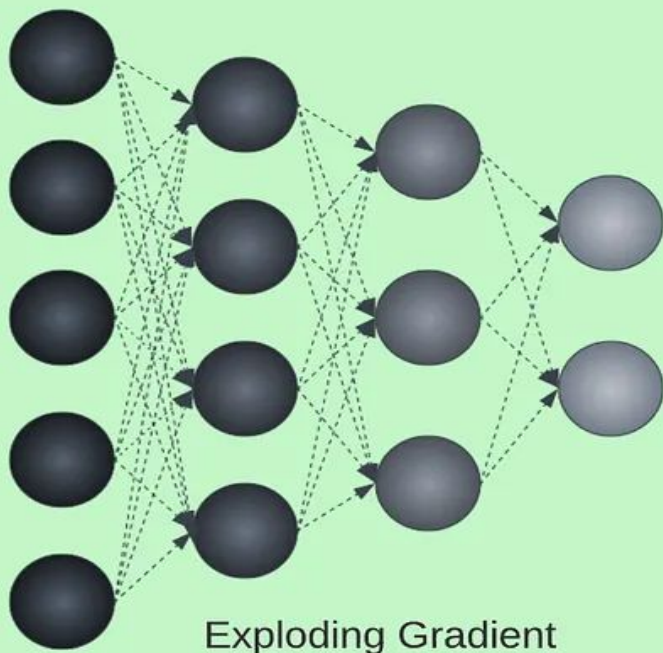


- **The Exploding Gradient Mechanics**
- If network weights are initialized with large values, or if sequential processing scales up values consecutively, the repeated matrix multiplications can quickly amplify numbers. A network multiplying values slightly greater than 1 over dozens of layers results in mathematical overflow. The gradient step becomes so massive that the optimization algorithm completely misses the global minimum and diverges.

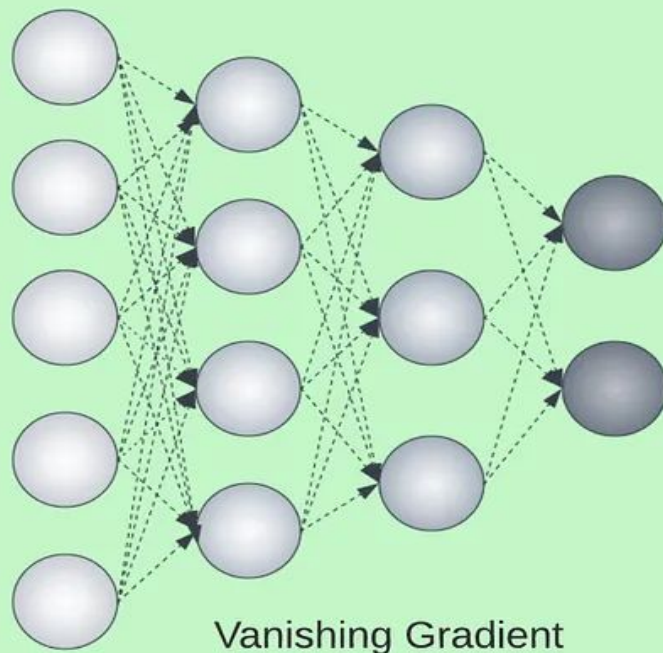


Comparison of the Two Problems

Feature	Vanishing Gradient Problem	Exploding Gradient Problem
Core Issue	Gradients become infinitely small ($\rightarrow 0$).	Gradients grow exponentially large ($\rightarrow \infty$).
Primary Cause	Saturating activation functions (Sigmoid, Tanh).	Poor weight initialization or high learning rates.
Impact on Weights	Network weights change minimally or stop updating.	Network weights change drastically and violently.
Model Symptoms	Extremely slow training or early stagnation.	Unstable training, fluctuating loss, or NaN values.
Affected Layers	Early layers closest to the input data.	Early layers in deep networks and long recurrent steps.



Exploding Gradient



Vanishing Gradient



Gradient



Backpropagation

- During the backpropagation algorithm, gradients are computed and propagated backward through the layers of a neural network.
- The vanishing gradient problem occurs when these gradients become extremely small as they propagate backward through multiple layers. Conversely, the exploding gradient problem happens when gradients grow exponentially during backpropagation.



Parameter	Meaning	Main Purpose	Effect on Neural Network	Typical Issue/Benefit
Learning Rate	Controls how large a step the optimizer takes while updating weights.	Controls the speed of learning.	High learning rate → faster but may overshoot; Low → slower but more stable.	Choosing the right value is important for convergence.
Gradient Descent	An optimization algorithm that updates weights in the direction that reduces the loss function.	Minimizes the loss/error.	Iteratively changes model parameters using gradients.	May be slow or get stuck depending on the optimization landscape.
Momentum	Uses a fraction of previous updates along with the current gradient.	Speeds up convergence and reduces oscillations.	Helps the optimizer continue moving in a consistent direction.	Faster and smoother training.
Regularization	Adds a penalty to the loss function for overly complex models.	Prevents overfitting.	Encourages smaller or simpler model weights.	Includes L1 and L2 regularization.

Sparsity	Encourages many model weights or neuron activations to become zero or inactive.	Creates a simpler and more efficient model.	Reduces the number of active connections or features.	Can reduce memory and computation requirements.
Vanishing Gradient	Gradients become extremely small during backpropagation.	It is a training problem, not an optimization technique.	Earlier layers learn very slowly or stop learning.	Common in deep networks and traditional RNNs; often associated with sigmoid/tanh activations.
Exploding Gradient	Gradients become extremely large during backpropagation.	It is a training problem.	Weight updates become excessively large and unstable.	Can cause numerical instability or model divergence.



Gradient Descent is the basic optimization method. Learning Rate controls its step size, while Momentum improves its movement using past updates. Regularization controls model complexity to reduce overfitting, and Sparsity encourages fewer active parameters. Vanishing and Exploding Gradients are problems that can occur during neural network training and affect the effectiveness of backpropagation.

